

Аннотация

Выпускная квалификационная работа посвящена повышению эффективности анализа и планирования транспортных маршрутов посредством разработки информационной системы моделирования транспортной сети на основе графовых алгоритмов. **Объектом исследования** являются информационные системы и технологии моделирования транспортных сетей на примере компании ООО «Энергомикс». **Предметом исследования** выступают графовые алгоритмы, математические методы и программные средства, применяемые для анализа и оптимизации транспортных маршрутов. В работе проведён анализ методов представления транспортных сетей в виде графовых моделей и алгоритмов решения задачи маршрутизации транспорта (VRP). Разработаны графовая модель транспортной сети и алгоритмы оптимизации маршрутов с учётом вместимости транспортных средств, временных окон доставки и тарифов перевозчиков. Программная реализация выполнена на языке Python с использованием FastAPI, PostgreSQL/PostGIS, NetworkX и Google OR-Tools. Экспериментальная апробация на данных компании показала снижение транспортных расходов на 8,4 %, общего пробега на 20,5 % и сокращение времени планирования с 4 часов до 2 минут на регион, что соответствует расчётной годовой экономии около 2,5 млн рублей.

Работа изложена на 72 страницах текста, содержит 8 рисунков, 4 приложения и список использованных источников из 30 наименований.

Ключевые слова: транспортная логистика, маршрутизация транспорта, графовые алгоритмы, информационная система, оптимизация маршрутов, задача VRP, кратчайший путь, FastAPI, PostGIS, OpenStreetMap, OR-Tools, кластеризация клиентов.

Оглавление

введение 5

1 исследование методов моделирования транспортных сетей 9

1.1 особенности организации транспортных сетей и маршрутов 9

1.2 представление транспортной сети в виде графовой модели 12

1.3 методы анализа и построения маршрутов в транспортных системах
15

1.4 обзор существующих программных решений для моделирования
транспортных маршрутов 20

1.5 определение требований к информационной системе моделирования
и анализа маршрутов 24

1.6 обоснование выбора технологий и инструментальных средств
разработки информационной системы 26

2 проектирование информационной системы моделирования и анализа
транспортных маршрутов на основе графовых алгоритмов 31

2.1 концепция и структура информационной системы 31

2.2 проектирование модели транспортной сети 34

2.3 проектирование структуры хранения данных о транспортных узлах
и маршрутах 36

2.4 разработка алгоритмов анализа транспортных маршрутов 40

2.4.1 алгоритм поиска кратчайшего пути 40

2.4.2 алгоритм оценки альтернативных маршрутов 41

2.5 проектирование пользовательского интерфейса 43

3 разработка и реализация информационной системы моделирования и
анализа транспортных маршрутов на основе графовых алгоритмов 47

3.2 реализация алгоритмов анализа маршрутов 49

3.3 реализация механизмов визуализации транспортных маршрутов 51

3.4 реализация информационной системы в целом 53

заключение 60

список использованных источников 63

приложение А 66

приложение Б 68

приложение В 70

приложение Г 72

введение

В современных условиях развития экономики и усиления конкурентной борьбы на рынке эффективность логистических процессов является одним из ключевых факторов конкурентоспособности предприятий любого масштаба. Транспортная логистика занимает центральное место в цепочке поставок, обеспечивая своевременное и экономически выгодное движение товаров от производителей к конечным потребителям. В условиях постоянного роста объемов перевозок и повышения требований клиентов к скорости и качеству доставки, оптимизация маршрутов доставки позволяет существенно снизить издержки и повысить качество обслуживания клиентов, что особенно актуально в условиях растущей конкуренции на рынке транспортных услуг. Согласно современным исследованиям в области транспортной логистики и маршрутизации, применение алгоритмов оптимизации и специализированных информационных систем позволяет снижать транспортные издержки, сокращать время планирования и повышать качество обслуживания клиентов [1, 2, 3]. В условиях растущей конкуренции и повышения требований потребителей к скорости доставки автоматизация процессов маршрутизации становится необходимым условием повышения эффективности логистической деятельности предприятий. Современные информационные технологии открывают принципиально новые возможности для решения сложных задач транспортной логистики, позволяя вычислять оптимальные маршруты за приемлемое время с учетом множества ограничений и требований. Использование графовых алгоритмов, методов дискретной оптимизации и имитационного моделирования позволяет находить решения, близкие к оптимальным, для задач, которые ранее считались практически неразрешимыми из-за их вычислительной сложности. Развитие вычислительной техники и появление эффективных программных библиотек оптимизации делают эти методы доступными для широкого круга предприятий. Несмотря на наличие коммерческих систем планирования транспорта, их применение в условиях средних предприятий часто ограничено высокой стоимостью внедрения, избыточной функциональностью и недостаточной гибкостью при настройке под специфику бизнес-процессов конкретной компании. Это обуславливает необходимость разработки собственных информационных систем, ориентированных на конкретные

задачи и особенности компании, с использованием современных открытых программных средств и библиотек.

Цель работы: повышение эффективности анализа и планирования транспортных маршрутов посредством разработки информационной системы моделирования транспортной сети, использующей графовые алгоритмы для поиска, оценки и сравнения возможных путей перемещения. Для достижения поставленной цели в работе решаются следующие

задачи:

- рассмотреть методы представления транспортных сетей в виде графовых моделей и определить их особенности при решении задач маршрутизации;
- выполнить обзор современных программных решений и сервисов, применяемых для анализа и построения транспортных маршрутов;
- выделить основные параметры транспортной сети, необходимые для моделирования маршрутов;
- разработать модель транспортной сети, обеспечивающую представление маршрутов и связей между транспортными узлами;
- спроектировать структуру информационной системы и механизм обработки данных о транспортных маршрутах;
- реализовать программные средства построения и анализа маршрутов с использованием графовых алгоритмов;
- обеспечить визуализацию транспортной сети и результатов поиска маршрутов в информационной системе.

Объектом исследования являются информационные системы и технологии моделирования транспортных сетей, обеспечивающие анализ, поиск и сравнение маршрутов на основе графовых алгоритмов на примере компании ООО «Энергомикс».

Предметом исследования являются математические методы, графовые алгоритмы и программные средства оптимизации, применяемые для моделирования и анализа транспортных маршрутов в условиях многокритериальной оптимизации с учетом ограничений по вместимости транспорта, временным окнам доставки и географическому покрытию

перевозчиков. В работе применяются методы системного анализа, теории графов и сетей, математического моделирования, дискретной оптимизации, объектно-ориентированного программирования, проектирования баз данных и современных информационных технологий. Для разработки программного обеспечения используются язык программирования Python, фреймворк FastAPI, система управления базами данных PostgreSQL с расширением PostGIS, библиотеки NetworkX и Google OR-Tools.

Практическая значимость работы заключается в создании программного комплекса, позволяющего повысить эффективность анализа и планирования транспортных маршрутов за счет автоматизации расчетов, сокращения времени подготовки маршрутов, снижения транспортных расходов, уменьшения общего пробега и повышения обоснованности выбора маршрутов и перевозчиков. Результаты работы могут быть применены не только в компании ООО «Энергомикс», но и в других организациях с аналогичными задачами транспортной логистики.

1 исследование методов моделирования транспортных сетей

1.1 особенности организации транспортных сетей и маршрутов

Транспортная сеть представляет собой сложную организационно-техническую систему, включающую совокупность транспортных узлов (представительств, складов, терминалов, распределительных центров) и транспортных связей (маршрутов, дорог, линий коммуникации) между ними. Эффективное функционирование транспортной сети определяется правильной организацией всех ее элементов, оптимальным распределением транспортных потоков и рациональным использованием транспортных ресурсов [1].

Компания ООО «Энергомикс» имеет разветвленную сеть представительств, расположенных в крупных городах Центрального федерального округа Российской Федерации. Клиентская база компании насчитывает более 1500 активных клиентов, распределенных по различным регионам Центральной России. Основные представительства компании расположены в следующих городах: Курск, Липецк, Воронеж, Тула и

Белгород. Каждое представительство обслуживает клиентов в радиусе до 300 километров, что определяет географию транспортной сети компании.

Анализ данных компании, проведенный в рамках настоящего исследования, показывает, что транспортная сеть имеет следующие характерные особенности: территориальная разрозненность клиентов, неравномерность заказов во времени, различные требования к срокам доставки, ограничения по грузоподъемности транспортных средств, необходимость учета тарифов различных перевозчиков, сезонные колебания спроса на транспортные услуги.

Территориальная разрозненность клиентов создает существенные сложности при планировании маршрутов, так как необходимо учитывать географическое расположение каждой точки доставки. Клиенты распределены неравномерно по территории обслуживания: в некоторых районах плотность клиентов высокая (городские агломерации), в других областях точки доставки находятся на значительном расстоянии друг от друга (сельская местность). Это требует применения методов кластеризации для группировки близкорасположенных клиентов и формирования эффективных маршрутов доставки.

Неравномерность заказов во времени проявляется в сезонных колебаниях спроса, пиковых нагрузках в конце месяца и перед праздничными днями. Анализ статистики заказов показывает, что в летний период объем доставок снижается на 20–25 процентов по сравнению с осенне-зимним периодом, когда наблюдается пик строительной активности. Такая неравномерность требует гибкого подхода к планированию маршрутов и резервирования транспортных мощностей для обработки пиковых нагрузок.

Различные требования к срокам доставки обусловлены спецификой бизнеса клиентов. Некоторые клиенты, работающие по системе точно-в-срок (just-in-time), требуют доставку в день заказа, другие согласны ждать 2–3 дня. Временные окна доставки также варьируются: некоторые клиенты принимают грузы только в утренние часы, другие — только после обеда. Эти требования должны учитываться при формировании маршрутов и распределении заказов по рейсам [2].

Ограничения по грузоподъемности транспортных средств являются критическим фактором при планировании доставки. Компания сотрудничает с различными перевозчиками, имеющими в своем парке транспорт разной вместимости: от небольших фургонов (до 1,5 тонн) до большегрузных автомобилей (до 20 тонн). Правильное распределение заказов по транспортным средствам с учетом их вместимости позволяет минимизировать количество рейсов и снизить транспортные расходы.

Учет тарифов различных перевозчиков является важной задачей при выборе оптимального маршрута. Каждый перевозчик имеет свою систему тарификации, которая может включать базовую ставку за рейс, ставку за километр пробега, минимальную стоимость заказа, дополнительные сборы за погрузку-разгрузку. Система должна учитывать все эти факторы при расчете стоимости доставки и выборе оптимального перевозчика для каждого маршрута.

К основным параметрам транспортной сети, необходимым для моделирования и анализа маршрутов в рамках данной работы, относятся: состав и расположение транспортных узлов, географические координаты точек доставки, расстояния и время перемещения между узлами, пропускные и эксплуатационные характеристики транспортных средств, временные окна обслуживания клиентов, приоритеты доставки, тарифные условия перевозчиков и ограничения зон обслуживания. Выделение указанных параметров является основой для последующего проектирования модели транспортной сети и информационной системы, ориентированной на повышение эффективности планирования маршрутов [3].

1.2 представление транспортной сети в виде графовой модели

Графовые модели являются наиболее распространенным и эффективным математическим аппаратом для представления и анализа транспортных сетей [4, 5]. В теории графов транспортная сеть представляется в виде графа, вершины которого соответствуют транспортным узлам (пунктам отправления, назначения и промежуточным точкам), а ребра — транспортным связям (дорогам, маршрутам) между ними.

Формально граф транспортной сети определяется как упорядоченная пара $G = (V, E)$, где V — конечное множество вершин (транспортных узлов), E — множество ребер (транспортных связей). Каждое ребро $e \in E$ имеет вес $w(e)$, характеризующий стоимость, время или расстояние прохождения данного участка сети. В общем случае вес может быть векторным, включающим несколько характеристик [6].

Для транспортных сетей характерно использование взвешенных графов, где каждому ребру приписывается один или несколько весовых коэффициентов. Наиболее часто используются следующие веса: расстояние между узлами в километрах, время прохождения в минутах с учетом скоростного режима дороги, стоимость проезда с учетом топлива и дорожных сборов. В многокритериальных задачах каждое ребро имеет вектор весов, и оптимизация выполняется по нескольким критериям одновременно [29].

В зависимости от структуры транспортной сети используются различные типы графов. Ориентированные графы (орграфы) применяются для моделирования сетей с односторонним движением, где направление ребра имеет существенное значение. Неориентированные графы используются для двусторонних дорог, где направление движения не ограничено. Смешанные графы применяются для комбинированных сетей, содержащих как односторонние, так и двусторонние участки.

Для компании ООО «Энергомикс» разработана детальная графовая модель транспортной сети, учитывающая специфику ее бизнес-процессов и особенности организации доставки. Модель включает следующие типы вершин: депо (представительства компании), клиенты (точки доставки), транзитные узлы (пересечения дорог, важные ориентиры).

Вершина-депо характеризуется следующими атрибутами: уникальный идентификатор, наименование представительства, географические координаты (широта и долгота в формате WGS84), почтовый адрес, контактная информация (телефон, email), график работы (часы приема и отправки грузов), максимальный радиус доставки. Депо являются начальными и конечными точками всех маршрутов доставки в своей зоне обслуживания.

Вершина-клиент имеет расширенный набор атрибутов: идентификатор клиента, наименование организации, юридический и фактический адрес, географические координаты места доставки, контактное лицо и телефон для связи, временное окно доставки (начало и конец интервала), приоритет обслуживания (обычный, высокий, критичный), специальные требования к доставке (наличие пандуса, необходимость манипулятора, особые условия хранения). Координаты клиентов используются для построения матрицы расстояний и определения ближайшего представительства.

Ребра графа представляют дорожные связи между вершинами. Каждое ребро имеет следующие весовые характеристики: расстояние в километрах, время в пути в минутах с учетом типа дороги, тип дороги (магистраль, основная дорога, второстепенная дорога, проселочная дорога). Данные о дорожной сети получены из открытого картографического проекта OpenStreetMap [23], который предоставляет бесплатный доступ к детальной картографической информации по всему миру.

Использование теории графов в задачах транспортной логистики обусловлено возможностью формального описания структуры транспортной сети и применения алгоритмов поиска, сравнения и оптимизации маршрутов [7]. Для целей данной работы графовая модель рассматривается не как абстрактная математическая конструкция, а как основа информационного представления транспортных узлов, связей между ними и параметров перемещения, используемых при автоматизированном планировании маршрутов.

Применительно к транспортным сетям графовые модели позволяют формализовать представление о структуре сети и процессах, происходящих в ней. Вершины графа соответствуют пунктам отправления, назначения и промежуточным точкам, а ребра — транспортным связям между ними. Веса ребер могут отражать различные характеристики: расстояние, время в пути, стоимость проезда, пропускную способность.

Важным преимуществом графовых моделей является возможность применения хорошо изученных алгоритмов для решения практических задач [30]. Алгоритмы поиска кратчайших путей, построения остовных деревьев, определения максимального потока имеют полиномиальную сложность и

эффективно работают даже для графов большого размера. Это делает графовые модели привлекательным инструментом для анализа и оптимизации транспортных сетей.

Развитие вычислительной техники и появление специализированных программных библиотек для работы с графами существенно расширило возможности практического применения графовых моделей. Современные библиотеки, такие как NetworkX для Python [8], Boost Graph Library для C++, JGraphT для Java, предоставляют эффективные реализации широкого набора алгоритмов на графах и удобные средства визуализации.

1.3 методы анализа и построения маршрутов в транспортных системах

Для анализа транспортных сетей широко применяются различные алгоритмы на графах. Алгоритм Дейкстры используется для поиска кратчайшего пути от одной вершины до всех остальных в графе с неотрицательными весами ребер. Алгоритм был разработан Эдсгером Дейкстрой в 1959 году [9] и является одним из наиболее известных алгоритмов для решения задачи о кратчайшем пути.

Алгоритм Дейкстры работает по жадному принципу: на каждом шаге выбирается вершина с минимальным текущим расстоянием от источника, и расстояния до ее соседей обновляются. Сложность алгоритма составляет $O((V + E) \log V)$ при использовании бинарной кучи, где V — количество вершин, E — количество ребер графа. Для транспортных сетей алгоритм Дейкстры применяется для построения матрицы расстояний между всеми парами точек доставки.

Алгоритм Флойда-Уоршелла находит кратчайшие пути между всеми парами вершин графа [10]. Алгоритм был независимо разработан Робертом Флойдом и Стивеном Уоршеллом в 1962 году. Временная сложность алгоритма составляет $O(V^3)$, что делает его применимым для графов среднего размера. Алгоритм использует метод динамического программирования и работает с матрицей смежности графа.

Алгоритм A^* (A-star) применяется для эвристического поиска пути с учетом дополнительной информации о целевой вершине, что позволяет

существенно ускорить поиск в больших графах. Алгоритм был разработан в 1968 году Питером Хартом, Нильсом Нильссоном и Бертрамом Рафаэлем [11]. Алгоритм использует эвристическую функцию $h(n)$, оценивающую стоимость пути от текущей вершины n до целевой. Если эвристика допустима (не переоценивает реальную стоимость), алгоритм A^* гарантирует нахождение оптимального пути.

Для графов с отрицательными весами ребер (но без отрицательных циклов) применяется алгоритм Беллмана-Форда, имеющий сложность $O(VE)$. Алгоритм был разработан Ричардом Беллманом и Лестером Фордом в 1958 году. В отличие от алгоритма Дейкстры, алгоритм Беллмана-Форда может обрабатывать ребра с отрицательными весами, что может быть полезно при моделировании транспортных сетей с учетом различных факторов.

Сравнительный анализ времени работы рассмотренных алгоритмов поиска кратчайших путей при различном числе вершин графа представлен на Рисунке 1.1. Из графика видно, что алгоритм Флойда-Уоршелла демонстрирует существенный рост времени работы при увеличении размера графа, тогда как алгоритмы Дейкстры и A^* сохраняют приемлемую производительность даже для больших графов.

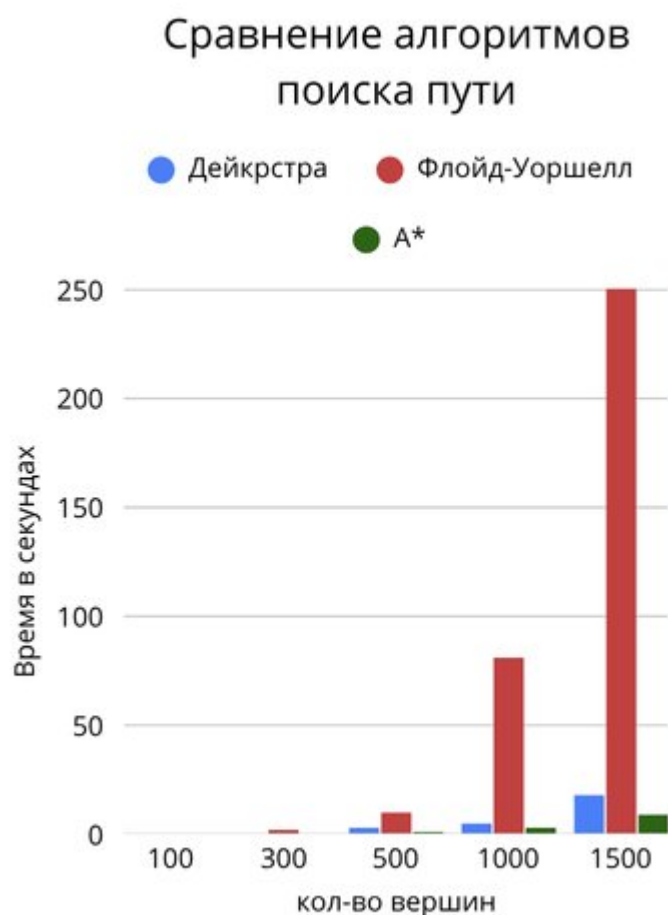


Рисунок 1.1 — Сравнение времени работы алгоритмов поиска кратчайших путей

Сравнение показывает, что для построения матрицы расстояний на графе транспортной сети с числом вершин до нескольких тысяч предпочтительным является алгоритм Дейкстры. Алгоритм A* может быть эффективен в задачах поиска одного маршрута между конкретной парой точек, однако при необходимости вычисления всех попарных расстояний его преимущество перед Дейкстрой нивелируется.

Задача маршрутизации транспорта (Vehicle Routing Problem, VRP) является одной из центральных задач транспортной логистики и комбинаторной оптимизации [12, 14]. Формально задача VRP формулируется следующим образом: дано множество клиентов с известными потребностями в доставке, необходимо определить набор маршрутов транспортных средств, минимизирующий суммарную стоимость при соблюдении всех ограничений.

Классическая задача VRP, также известная как CVRP (Capacitated Vehicle Routing Problem), имеет следующие ограничения: каждый клиент должен быть обслужен ровно одним транспортным средством, каждое транспортное средство начинает и заканчивает маршрут в депо, суммарный спрос клиентов на маршруте не должен превышать вместимость транспортного средства. Целевая функция обычно представляет собой суммарное расстояние всех маршрутов или общую стоимость перевозок [15].

Существует множество вариаций задачи VRP, учитывающих дополнительные факторы и ограничения, характерные для реальных логистических систем. Задача VRP с временными окнами (VRPTW - Vehicle Routing Problem with Time Windows) добавляет ограничения на время посещения каждого клиента: каждый клиент должен быть обслужен в определенный интервал времени. Это существенно усложняет задачу, так как необходимо отслеживать время прибытия к каждому клиенту [3].

Для решения задачи VRP применяются различные методы, которые можно разделить на три основные категории: точные методы, эвристические методы и метаэвристические методы. Точные методы гарантируют нахождение оптимального решения, но применимы только для задач небольшого размера. К ним относятся метод ветвей и границ, метод динамического программирования, методы целочисленного линейного программирования [16, 17, 18].

Эвристические методы не гарантируют оптимальности, но позволяют находить допустимые решения за приемлемое время даже для задач большого размера. Наиболее известной эвристикой для VRP является алгоритм Кларка-Райта (Savings Algorithm), предложенный в 1964 году [13]. Алгоритм основан на идее объединения маршрутов для экономии расстояния. Экономия при объединении маршрутов, проходящих через клиентов i и j , вычисляется по формуле: $s(i,j) = d(0,i) + d(0,j) - d(i,j)$, где d — расстояние, 0 — депо.

Метаэвристические методы представляют собой высокоуровневые стратегии поиска, которые могут быть применены к различным задачам оптимизации. К наиболее известным метаэвристикам для VRP относятся: генетические алгоритмы, имитация отжига (simulated annealing), поиск с запретами (tabu search), муравьиные алгоритмы, алгоритмы роя частиц. Эти

методы позволяют находить качественные решения для задач большого размера за приемлемое время.

Имитационное моделирование представляет собой мощный инструмент анализа сложных систем, позволяющий исследовать поведение транспортной системы в различных условиях без проведения дорогостоящих реальных экспериментов. Метод основан на создании компьютерной модели, воспроизводящей функционирование реальной системы с заданной степенью детализации и точности.

В транспортной логистике имитационное моделирование применяется для решения широкого круга задач: оценки пропускной способности транспортной сети при различных сценариях загрузки, анализа влияния случайных факторов (пробки на дорогах, поломки транспорта, задержки при погрузке), тестирования различных стратегий управления транспортными потоками, обучения персонала работе в нештатных ситуациях, оценки эффективности предлагаемых изменений в логистической системе. Для решения задач настоящей ВКР в качестве базовых выбраны графовые алгоритмы поиска кратчайших путей и алгоритмы маршрутизации транспорта, поскольку именно они обеспечивают практическую применимость при разработке информационной системы моделирования и анализа маршрутов. Их использование позволяет сократить время вычисления допустимых решений, повысить обоснованность выбора маршрутов и тем самым обеспечить повышение эффективности анализа и планирования доставки в сравнении с ручным способом формирования маршрутов.

1.4 обзор существующих программных решений для моделирования транспортных маршрутов

На современном рынке информационных технологий представлено множество программных решений для оптимизации транспортных маршрутов, различающихся по функциональности, стоимости, сложности внедрения и другим характеристикам. Проведенный анализ позволил выделить следующие основные категории решений: корпоративные системы класса TMS, специализированные сервисы маршрутизации, открытые библиотеки оптимизации.

Корпоративные системы класса TMS (Transportation Management System) представляют собой комплексные решения, охватывающие все аспекты управления транспортом: планирование маршрутов, управление перевозчиками, отслеживание грузов, расчет стоимости перевозок, документооборот, аналитику. К наиболее известным решениям относятся: SAP Transportation Management, Oracle Transportation Management, 1С:Логистика. Управление транспортом.

SAP Transportation Management является частью экосистемы SAP и предоставляет широкий набор функций для планирования и оптимизации транспортных процессов. Система поддерживает различные режимы транспорта (автомобильный, железнодорожный, морской, воздушный), интеграцию с GPS-трекингом для отслеживания грузов в реальном времени, расчет стоимости перевозок с учетом множества факторов. Недостатками являются высокая стоимость внедрения (от нескольких миллионов рублей), длительный срок внедрения (6–18 месяцев), сложность настройки под специфические требования компании.

Oracle Transportation Management — еще одно корпоративное решение, предоставляющее функции планирования и оптимизации транспортных операций. Система интегрируется с другими продуктами Oracle и поддерживает сложные сценарии логистики. Основными недостатками являются высокая стоимость лицензий и необходимость значительных инвестиций в инфраструктуру и обучение персонала.

1С:Логистика. Управление транспортом — российское решение, интегрированное с платформой 1С. Система обеспечивает планирование перевозок, учет транспортных средств, расчет стоимости доставки, интеграцию с картографическими сервисами. Преимуществами являются соответствие российскому законодательству, интеграция с другими продуктами 1С, русскоязычная поддержка. Недостатками являются ограниченные возможности оптимизации маршрутов по сравнению с зарубежными аналогами.

Специализированные сервисы маршрутизации фокусируются на конкретной задаче оптимизации маршрутов и обычно предоставляются по модели SaaS (Software as a Service), что снижает начальные затраты на

внедрение. Примерами таких сервисов являются: Яндекс.Маршрутизация, 2ГИС Логистика, LogiNext, OptimoRoute, Route4Me.

Яндекс.Маршрутизация — российский сервис, использующий данные Яндекс.Карт для построения оптимальных маршрутов. Сервис учитывает пробки в реальном времени, временные окна клиентов, вместимость транспорта, особенности дорожной сети. Преимуществами являются актуальные картографические данные по России и странам СНГ, русскоязычный интерфейс и техническая поддержка, доступная стоимость (от 5000 рублей в месяц). Недостатком является ограниченная возможность интеграции с внешними системами.

2ГИС Логистика — сервис, основанный на данных справочника 2ГИС. Сервис позволяет планировать маршруты с учетом временных окон, вместимости транспорта, загруженности дорог. Преимуществами являются детальные данные по организациям, включая часы работы, актуальная информация о дорожной сети. Недостатками являются ограниченная география (в основном Россия и страны СНГ), ограниченные возможности кастомизации.

LogiNext — международный сервис для оптимизации логистики и управления доставкой. Сервис предоставляет функции планирования маршрутов, отслеживания грузов в реальном времени, аналитики и отчетности. Преимуществами являются широкая география покрытия, интеграция с различными системами, мобильное приложение для водителей. Недостатками являются высокая стоимость для российских компаний, необходимость адаптации под локальные условия.

Открытые библиотеки оптимизации позволяют разрабатывать собственные решения на основе проверенных алгоритмов, обеспечивая полный контроль над функциональностью и возможность бесплатного использования. Наиболее известными являются: Google ORTools [19], NetworkX [8], OSRM (Open Source Routing Machine), Vroom, jsprit.

Google ORTools — мощная библиотека оптимизации с открытым исходным кодом, разработанная компанией Google. Библиотека включает эффективные алгоритмы для решения различных задач оптимизации, включая VRP, TSP, задачи расписания, задачи удовлетворения ограничений.

Для решения VRP используются методы constraint programming и метаэвристики, позволяющие находить качественные решения за приемлемое время. Библиотека имеет интерфейсы для Python, C++, Java и C#.

NetworkX — библиотека Python для создания, манипулирования и изучения структуры, динамики и функций сложных сетей. Библиотека предоставляет стандартные алгоритмы для работы с графами: поиск кратчайших путей, построение минимальных остовных деревьев, поиск максимального потока, анализ центральности вершин. NetworkX хорошо интегрируется с другими библиотеками Python для научных вычислений.

OSRM (Open Source Routing Machine) — высокопроизводительный движок для построения маршрутов на основе данных OpenStreetMap. OSRM использует современные алгоритмы для быстрого поиска кратчайших путей и может обрабатывать миллионы запросов в секунду. Движок поддерживает различные профили маршрутизации (автомобиль, велосипед, пешеход) и учитывает ограничения дорожного движения.

На Рисунке 1.2 представлено сравнение функциональности рассмотренных систем по ключевым параметрам, актуальным для задач компании ООО «Энергомикс». Оценка выполнялась по пятибалльной шкале на основе анализа технической документации, демонстрационных версий и публикаций пользователей.

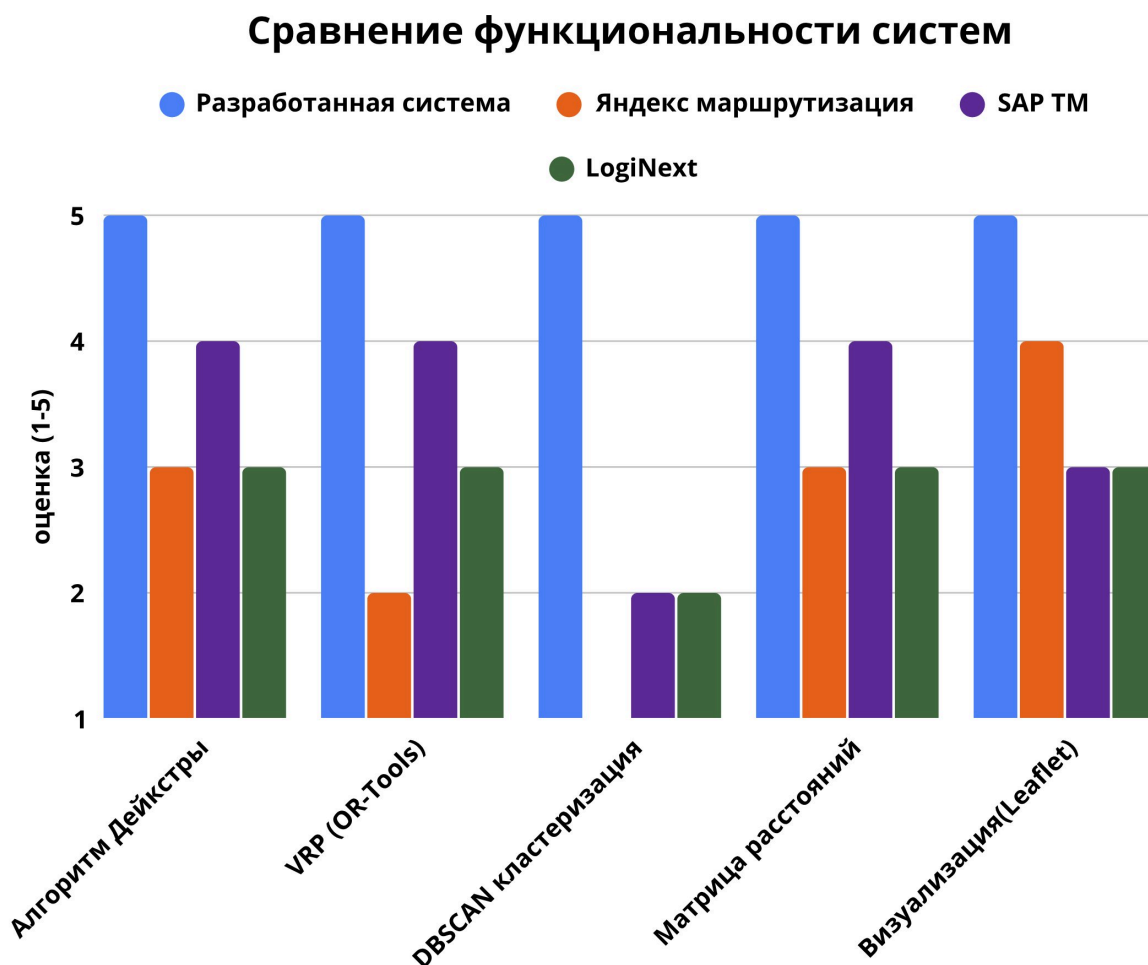


Рисунок 1.2 — Сравнение функциональности систем оптимизации маршрутов

Сопоставление систем показывает, что разработанная информационная система, ориентированная на конкретные требования ООО «Энергомикс», по совокупности рассмотренных характеристик не уступает коммерческим решениям и обеспечивает преимущество в части интеграции необходимых алгоритмов (Дейкстры, VRP, DBSCAN-кластеризации) в едином программном комплексе.

1.5 определение требований к информационной системе моделирования и анализа маршрутов

На основе анализа особенностей транспортной сети ООО «Энергомикс» и возможностей существующих программных решений сформулированы требования к разрабатываемой информационной системе.

Требования разделены на функциональные, описывающие набор реализуемых функций, и нефункциональные, определяющие характеристики качества системы.

К функциональным требованиям относятся следующие. Управление справочными данными должно обеспечивать ввод, редактирование, удаление и просмотр информации о клиентах, представительствах и перевозчиках, включая их географические координаты, контактные данные, тарифы и зоны покрытия.

Построение матрицы расстояний должно выполняться на основе реальных дорожных данных OpenStreetMap [23] с учетом типа дорог и направления движения. Матрица должна сохраняться для повторного использования и инвалидироваться при изменении набора клиентов или дорожной сети.

Кластеризация клиентов должна выполняться по географическим координатам с использованием алгоритма DBSCAN. Параметры кластеризации (радиус соседства, минимальное число точек) должны быть настраиваемыми пользователем.

Оптимизация маршрутов должна решать задачу VRP с ограничениями вместимости и временными окнами доставки. Должны поддерживаться выбор критерия оптимизации (расстояние, время, стоимость), задание ограничений по числу транспортных средств и их вместимости.

Выбор перевозчика должен осуществляться по минимальной стоимости доставки с учетом тарифов и зон покрытия. Должна быть предусмотрена возможность ручной корректировки автоматического выбора.

Визуализация результатов должна включать отображение представительств, клиентов и сформированных маршрутов на интерактивной карте с возможностью масштабирования, фильтрации и просмотра детальной информации.

Интеграция с внешними системами. Система должна обеспечивать обмен данными с существующей системой управления транспортом (TMS) компании через API. Должны поддерживаться импорт клиентской базы, экспорт сформированных маршрутов, синхронизация статусов доставки.

К нефункциональным требованиям относятся следующие. Производительность: время построения матрицы расстояний для 1500 клиентов не должно превышать 5 минут, время оптимизации маршрутов для одного региона (до 300 клиентов) — 30 секунд, время отклика интерфейса — 1 секунды.

Надежность: система должна обеспечивать сохранность данных при сбоях, поддерживать резервное копирование базы данных, обеспечивать корректное восстановление после отказов оборудования. Безопасность: должна быть реализована аутентификация пользователей, разграничение прав доступа по ролям, журналирование действий пользователей. Масштабируемость: система должна обеспечивать возможность работы при увеличении числа клиентов до 5000, числа представительств — до 20, числа перевозчиков — до 50.

1.6 обоснование выбора технологий и инструментальных средств разработки информационной системы

В качестве основного языка программирования серверной части системы выбран Python. Выбор обусловлен наличием развитой экосистемы библиотек для анализа данных, моделирования графов и решения задач оптимизации, включая NetworkX [8], NumPy, SciPy и Google OR-Tools [19], а также высокой скоростью разработки и хорошей интеграцией с современными средствами построения веб-сервисов. Использование Python позволяет сократить трудоемкость реализации алгоритмической части системы и обеспечить удобство сопровождения программного кода.

Для разработки REST API рассматривались следующие фреймворки: Django REST Framework, Flask, FastAPI [20]. Django REST Framework является мощным и зрелым фреймворком, но имеет избыточную функциональность для данной задачи и меньшую производительность по сравнению с современными альтернативами. Flask — легковесный фреймворк, предоставляющий базовые возможности для создания API, но требующий ручной настройки многих компонентов (валидация, документация, асинхронность).

FastAPI был выбран в качестве фреймворка для разработки API по следующим причинам. Во-первых, FastAPI обеспечивает высокую производительность благодаря использованию асинхронной обработки запросов (asyncio) и современных возможностей Python (type hints). По бенчмаркам, FastAPI сопоставим по производительности с Node.js и Go, значительно превосходя Django и Flask. Во-вторых, FastAPI автоматически генерирует интерактивную документацию API (Swagger UI, ReDoc) на основе аннотаций типов. В-третьих, FastAPI имеет встроенную валидацию данных на основе Pydantic, что снижает количество ошибок и упрощает разработку.

Для хранения данных рассматривались следующие системы управления базами данных: PostgreSQL [21], MySQL, MongoDB. MySQL является популярной реляционной СУБД с открытым исходным кодом, но имеет ограниченную поддержку географических данных и пространственных запросов. MongoDB — документоориентированная NoSQL база данных, обеспечивающая гибкость схемы данных, но менее эффективна для сложных аналитических запросов и не поддерживает пространственные индексы на уровне PostgreSQL.

PostgreSQL была выбрана в качестве системы управления базами данных по следующим причинам. Во-первых, PostgreSQL — мощная объектно-реляционная СУБД с открытым исходным кодом, имеющая репутацию надежной и производительной системы. Во-вторых, PostgreSQL имеет расширение PostGIS [22], которое добавляет поддержку географических объектов и пространственных запросов, что критически важно для работы с транспортной сетью. PostGIS поддерживает пространственные индексы (R-tree, GiST), функции для расчета расстояний, определения пересечений и другие геометрические операции.

Для работы с графами и решения задачи маршрутизации были выбраны библиотеки NetworkX и Google OR-Tools. NetworkX — это библиотека Python для создания, манипулирования и изучения структуры, динамики и функций сложных сетей. Библиотека предоставляет стандартные алгоритмы для работы с графами: поиск кратчайших путей (алгоритмы Дейкстры [9], Беллмана-Форда, A* [11]), построение минимальных остовных деревьев, поиск максимального потока. NetworkX хорошо интегрируется с другими библиотеками Python для научных вычислений (NumPy, SciPy).

Google OR-Tools — мощная библиотека оптимизации с открытым исходным кодом, разработанная компанией Google. Библиотека включает эффективные алгоритмы для решения различных задач оптимизации, включая VRP, TSP (задача коммивояжера), задачи расписания, задачи удовлетворения ограничений. Для решения задачи VRP OR-Tools использует комбинацию методов constraint programming и метаэвристик (guided local search, simulated annealing, tabu search), позволяющих находить качественные решения за приемлемое время.

Для получения данных о дорожной сети используется проект OpenStreetMap (OSM) — открытый картографический проект, создаваемый сообществом добровольцев. Данные OSM распространяются под открытой лицензией Open Database License и предоставляют детальную информацию о дорожной сети, включая геометрию дорог, типы дорог, ограничения скорости, направления движения. Для получения данных из OSM используется Overpass API — специализированный язык запросов, позволяющий извлекать геопространственные данные.

Для разработки пользовательского интерфейса был выбран фреймворк React [24] — библиотека JavaScript для создания пользовательских интерфейсов, разработанная Facebook. Преимуществами React являются компонентный подход, позволяющий создавать переиспользуемые элементы интерфейса; виртуальный DOM, обеспечивающий высокую производительность при обновлении интерфейса; обширная экосистема библиотек и инструментов; большое сообщество разработчиков. Для управления состоянием приложения используется библиотека Redux.

Для отображения карт и визуализации транспортной сети используется библиотека Leaflet [25] — открытая JavaScript-библиотека для создания интерактивных карт. Leaflet предоставляет простой и гибкий API для отображения карт, добавления маркеров, линий, полигонов, всплывающих подсказок. Библиотека поддерживает различные источники картографических тайлов, включая OpenStreetMap, и имеет небольшой размер, что обеспечивает быструю загрузку страницы.

Таким образом, выбранный стек технологий (Python + FastAPI + PostgreSQL/PostGIS + NetworkX + OR-Tools + React + Leaflet) обеспечивает

оптимальное сочетание производительности, функциональности, гибкости и скорости разработки для решения задачи моделирования и анализа транспортных маршрутов.

Выводы по главе 1

В первой главе проведено исследование методов моделирования транспортных сетей и алгоритмов, применяемых для решения задач маршрутизации. Установлено, что наиболее адекватным способом представления транспортной сети в рамках поставленной темы является графовая модель, позволяющая формализовать транспортные узлы, связи между ними и параметры перемещения. Рассмотрены алгоритмы поиска кратчайших путей и подходы к решению задачи маршрутизации транспорта, показана их применимость для автоматизации процессов анализа и планирования доставки.

Выполненный обзор современных программных решений показал, что готовые корпоративные и облачные сервисы не в полной мере учитывают специфику логистических процессов ООО «Энергомикс», в связи с чем обоснован выбор разработки собственной информационной системы на основе открытых библиотек и технологий. В ходе анализа выделены основные параметры транспортной сети, необходимые для моделирования маршрутов, и сформулированы функциональные и нефункциональные требования к системе.

Результаты первой главы создают теоретическую и методическую основу для проектирования информационной системы моделирования и анализа транспортных маршрутов, ориентированной на повышение эффективности планирования за счет сокращения времени расчетов, снижения транспортных затрат и повышения обоснованности выбора маршрутов.

2 проектирование информационной системы моделирования и анализа транспортных маршрутов на основе графовых алгоритмов

2.1 концепция и структура информационной системы

Разрабатываемая информационная система построена по клиент-серверной архитектуре с разделением на уровень представления, уровень бизнес-логики и уровень данных. Выбор такой архитектуры обусловлен требованиями к масштабируемости, управляемости и надежности системы, а также необходимостью обеспечить эффективную обработку данных о транспортных маршрутах, быстрый расчет результатов и удобную интеграцию алгоритмических модулей в единый программный комплекс.

Уровень представления реализован в виде веб-приложения, предоставляющего пользовательский интерфейс для работы с системой. Frontend разработан с использованием современных веб-технологий: HTML5 для разметки, CSS3 для стилизации, JavaScript с фреймворком React [24] для реализации интерактивности. Для отображения карт используется библиотека Leaflet [25], обеспечивающая интерактивное отображение географических данных с возможностью масштабирования, перемещения и добавления маркеров.

Архитектура frontend построена на компонентном подходе, где каждый элемент интерфейса (карта, список клиентов, панель настроек, таблица маршрутов) представляет собой отдельный компонент с четко определенным интерфейсом. Это упрощает разработку, тестирование и повторное использование компонентов. Для управления состоянием приложения используется библиотека Redux, обеспечивающая предсказуемое поведение при работе с данными.

Уровень бизнес-логики реализован в виде REST API сервиса на языке программирования Python с использованием современного фреймворка FastAPI [20]. FastAPI обеспечивает высокую производительность благодаря использованию асинхронной обработки запросов, автоматическую генерацию интерактивной документации API (Swagger UI), валидацию входных данных

на основе type hints, поддержку современных стандартов OpenAPI и JSON Schema.

Основные компоненты backend включают следующие модули: модуль управления данными (CRUD операции для клиентов, представительств, перевозчиков), модуль построения матрицы расстояний (взаимодействие с OpenStreetMap, расчет кратчайших путей), модуль оптимизации маршрутов (интеграция с OR-Tools, решение задачи VRP), модуль выбора перевозчика (расчет стоимости, сравнение тарифов), модуль интеграции с внешними системами (API для обмена данными с TMS).

Уровень данных обеспечивает надежное хранение и эффективный доступ к информации системы. В качестве системы управления базами данных используется PostgreSQL [21] - мощная объектно-реляционная СУБД с открытым исходным кодом. Для поддержки географических данных используется расширение PostGIS [22], которое добавляет в PostgreSQL поддержку пространственных типов данных, пространственных индексов (R-tree, GiST) и функций для выполнения геометрических запросов.

Взаимодействие между уровнями осуществляется через стандартизированные интерфейсы. Frontend взаимодействует с backend через REST API по протоколу HTTP/HTTPS с использованием формата JSON для передачи данных. Backend обращается к базе данных через ORM (Object-Relational Mapping) с использованием библиотеки SQLAlchemy, которая обеспечивает абстракцию от конкретной СУБД и упрощает работу с данными.

Общая трехуровневая архитектура системы с указанием используемых технологий на каждом уровне представлена на Рисунке 2.1.

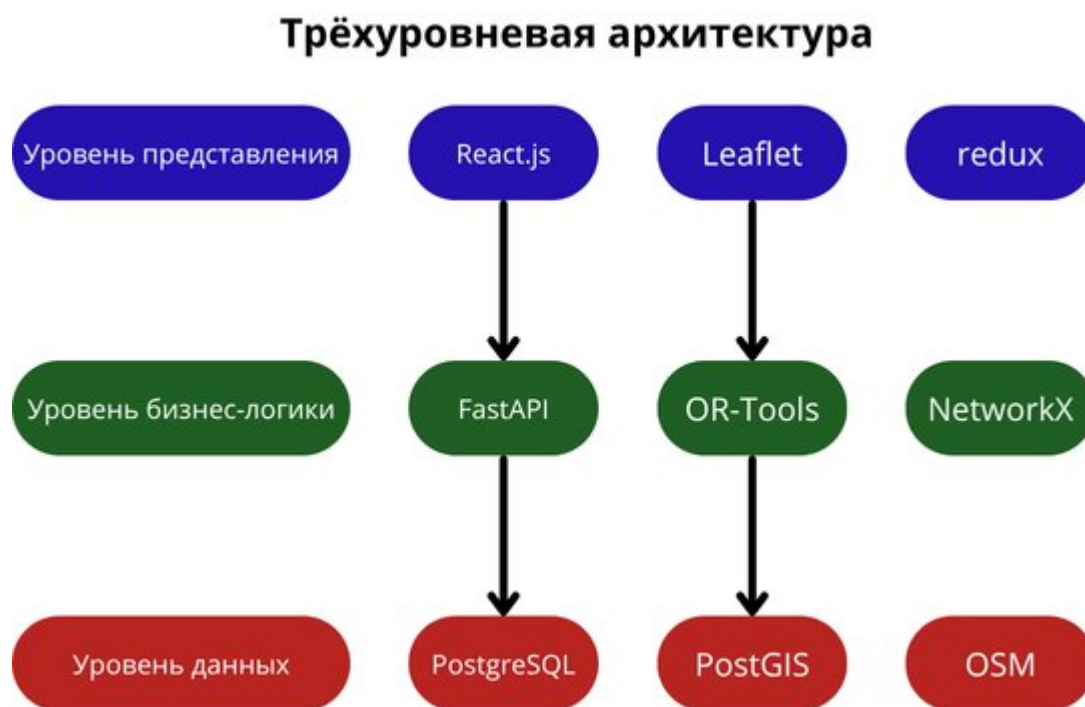


Рисунок 2.1 — Трёхуровневая архитектура информационной системы

Для оптимизации маршрутов используется библиотека Google OR-Tools [19], которая взаимодействует с backend через Python API. OR-Tools предоставляет эффективные алгоритмы для решения задачи VRP с учетом различных ограничений: вместимости транспорта, временных окон, приоритетов. Для получения дорожных данных используется сервис OpenStreetMap [23] через API Overpass. Overpass API позволяет выполнять сложные запросы к базе данных OpenStreetMap для получения информации о дорожной сети в заданном географическом регионе. Данные кэшируются для снижения нагрузки на внешний сервис и ускорения последующих запросов.

2.2 проектирование модели транспортной сети

В рамках проектирования информационной системы для ООО «Энергомикс» разработана графовая модель транспортной сети, ориентированная на автоматизированное построение и анализ маршрутов доставки. Модель включает три типа вершин: депо, клиентов и транзитные узлы дорожной сети [4, 7]. Такое представление обеспечивает формальное описание объектов маршрутизации и создает основу для вычисления расстояний, времени перемещения и оценки альтернативных путей.

Вершина-депо характеризуется следующими атрибутами: уникальный идентификатор, наименование представительства, географические координаты (широта и долгота в формате WGS84), почтовый адрес, контактная информация (телефон, email), график работы (часы приема и отправки грузов), максимальный радиус доставки. Депо являются начальными и конечными точками всех маршрутов доставки в своей зоне обслуживания.

Вершина-клиент имеет расширенный набор атрибутов: идентификатор клиента, наименование организации, юридический и фактический адрес, географические координаты места доставки, контактное лицо и телефон для связи, временное окно доставки (начало и конец интервала), приоритет обслуживания (обычный, высокий, критичный), специальные требования к доставке (наличие пандуса, необходимость манипулятора, особые условия хранения). Координаты клиентов используются для построения матрицы расстояний и определения ближайшего представительства.

Ребра графа представляют дорожные связи между вершинами. Каждое ребро имеет следующие весовые характеристики: расстояние в километрах, время в пути в минутах с учетом типа дороги, тип дороги (магистраль, основная дорога, второстепенная дорога, проселочная дорога). Данные о дорожной сети получены из открытого картографического проекта OpenStreetMap [23].

Математически транспортная сеть представляется как взвешенный граф $G = (V, E, W)$, где $V = \{v_1, v_2, \dots, v_n\}$ - множество вершин, $E \subseteq V * V$ - множество ребер, $W: E \rightarrow \mathbb{R}^+$ - функция весов ребер. Для многокритериальной оптимизации вес ребра представляется вектором $w(e) = (w_1(e), w_2(e), \dots, w_k(e))$, где k - количество критериев оптимизации (обычно расстояние, время, стоимость) [6].

Для построения матрицы расстояний между всеми парами точек сети используется алгоритм Дейкстры [9]. Матрица расстояний D размерности $n * n$, где n - количество точек сети, содержит длины кратчайших путей между парами точек: $D[i][j] = d(v_i, v_j)$, где $d(v_i, v_j)$ - длина кратчайшего пути от точки i до точки j . В частном случае неориентированного графа такая матрица является симметричной, однако для реальной дорожной сети с учетом

направлений движения и ограничений возможна несимметричность значений. Матрица расстояний используется как входной набор данных для алгоритмов оптимизации маршрутов.

Для предварительной группировки клиентов перед оптимизацией маршрутов используется алгоритм кластеризации DBSCAN (Density-Based Spatial Clustering of Applications with Noise). Алгоритм выделяет кластеры как области с высокой плотностью точек, разделенные областями с низкой плотностью. Основным преимуществом DBSCAN является способность находить кластеры произвольной формы и выделять выбросы (шумовые точки), которые не принадлежат ни одному кластеру [28].

Алгоритм DBSCAN использует два ключевых параметра: ϵ (epsilon) - максимальное расстояние между точками для включения в один кластер, $\min_samples$ - минимальное количество точек для формирования кластера. Для транспортной сети параметр ϵ выбирается исходя из вместимости транспортных средств и характерных расстояний между клиентами. Обычно ϵ составляет 20–30 километров. Результатом кластеризации является разбиение клиентов на группы, каждая из которых обслуживается отдельным маршрутом.

Для решения задачи маршрутизации транспорта (VRP) используется библиотека Google OR-Tools [19]. Математическая постановка задачи VRP для системы формулируется следующим образом. Дано: множество клиентов $C = \{1, 2, \dots, n\}$, множество депо $D = \{0_1, 0_2, \dots, 0_m\}$, матрица расстояний D , где $d[i][j]$ - расстояние от точки i до точки j , вместимость транспортного средства Q , спрос каждого клиента $q[i]$, временные окна доставки $[a_i, b_i]$ для каждого клиента i [12].

Требуется найти набор маршрутов $R = \{r_1, r_2, \dots, r_k\}$, минимизирующий суммарное расстояние, такой что: каждый клиент посещен ровно один раз; каждый маршрут начинается и заканчивается в одном из депо; суммарный спрос на маршруте не превышает Q ; время прибытия к клиенту попадает в его временное окно $[a_i, b_i]$.

2.3 проектирование структуры хранения данных о транспортных узлах и маршрутах

База данных системы спроектирована с учетом требований нормализации для исключения избыточности данных и обеспечения целостности, а также требований производительности для обеспечения быстрого доступа к часто запрашиваемым данным. Основные сущности базы данных и их атрибуты описаны ниже.

Сущность Клиент хранит полную информацию о клиентах компании. Таблица clients содержит следующие поля: id — уникальный идентификатор клиента (первичный ключ, целое число, автоинкремент), name — наименование организации (строка до 255 символов, обязательное поле), address — полный адрес доставки (строка до 500 символов), latitude — географическая широта (число с плавающей точкой, 8 знаков после запятой), longitude — географическая долгота (число с плавающей точкой, 8 знаков после запятой), phone — контактный телефон (строка до 20 символов), contact_person — ФИО контактного лица (строка до 100 символов), email — электронная почта (строка до 100 символов), time_window_start — начало временного окна доставки (время), time_window_end — конец временного окна доставки (время), priority — приоритет обслуживания (целое число: 1 — низкий, 2 — обычный, 3 — высокий, 4 — критичный), special_requirements — специальные требования к доставке (текст), is_active — флаг активности клиента (логическое значение), created_at — дата создания записи (timestamp), updated_at — дата последнего обновления (timestamp).

Географические координаты клиентов хранятся в виде типа POINT с использованием расширения PostGIS [22]. Это позволяет эффективно выполнять пространственные запросы, такие как поиск клиентов в заданном радиусе от точки, определение клиентов внутри полигона, расчет расстояний между точками с учетом сферичности Земли. Для ускорения пространственных запросов создается пространственный индекс GiST на поле с координатами.

Сущность Представительство хранит информацию о филиалах компании. Таблица depots содержит поля: id — уникальный идентификатор (первичный ключ), name — наименование представительства (строка до 255

символов), address — полный адрес (строка до 500 символов), latitude — широта (число с плавающей точкой), longitude — долгота (число с плавающей точкой), phone — контактный телефон, email — электронная почта, working_hours — режим работы (строка, например «08:00–18:00»), max_delivery_radius — максимальный радиус доставки в километрах (целое число), coverage_area — географическая зона покрытия (полигон в формате PostGIS), is_active — флаг активности.

Сущность Перевозчик хранит информацию о транспортных компаниях-партнерах. Таблица carriers содержит поля: id — уникальный идентификатор, name — наименование перевозчика, contact_person — контактное лицо, phone — телефон, email — электронная почта, vehicle_capacity — вместимость транспортного средства в кубических метрах (число), base_rate — базовая ставка за рейс в рублях, rate_per_km — ставка за километр пробега в рублях, min_charge — минимальная стоимость рейса, coverage_area — зона покрытия (полигон PostGIS), max_trip_distance — максимальное расстояние рейса, is_active — флаг активности.

Сущность Маршрут хранит информацию о сформированных маршрутах доставки. Таблица routes содержит поля: id — уникальный идентификатор маршрута, depot_id — идентификатор представительства (внешний ключ на таблицу depots), carrier_id — идентификатор перевозчика (внешний ключ на таблицу carriers), delivery_date — дата доставки, total_distance — общее расстояние маршрута в километрах, total_time — общее время маршрута в минутах, total_cost — стоимость доставки в рублях, vehicle_count — количество транспортных средств, status — статус маршрута (запланирован, в работе, выполнен, отменен), created_at — дата создания, optimized_at — дата оптимизации, created_by — пользователь, создавший маршрут.

Структурная схема основных таблиц базы данных и связей между ними представлена на Рисунке 2.2.

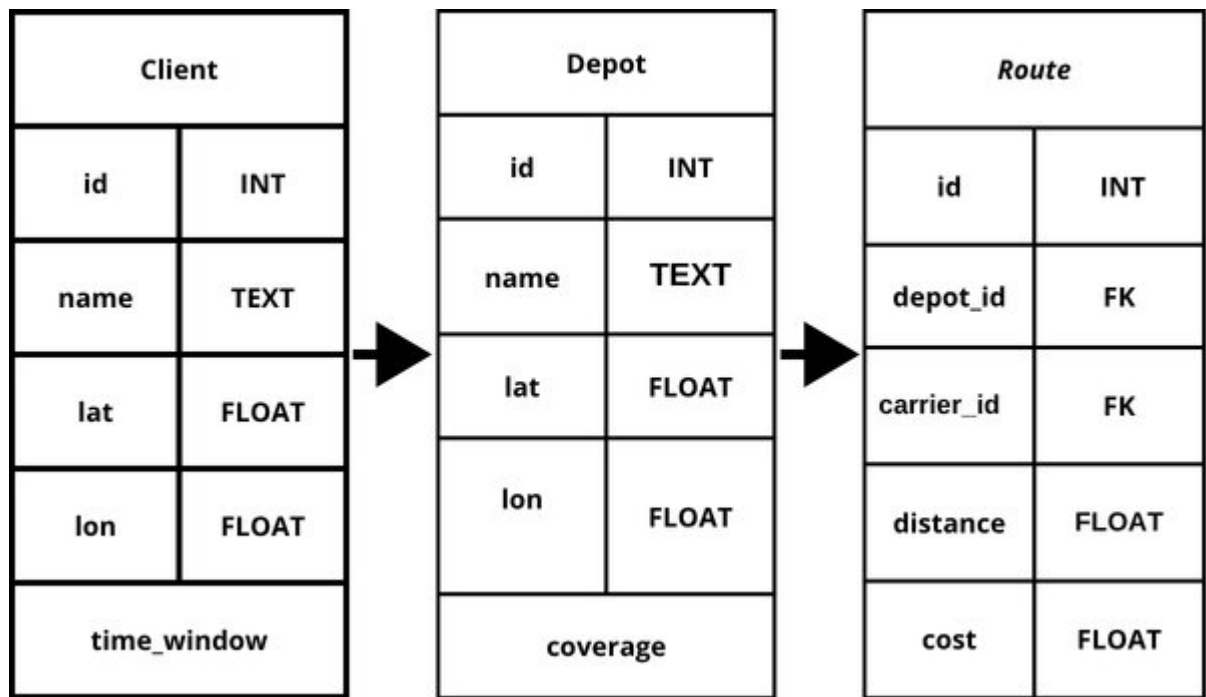


Рисунок 2.2 — Структура основных таблиц базы данных

Связи между таблицами реализуются через внешние ключи. Таблица routes имеет внешние ключи на таблицы depots и carriers. Таблица route_points (точки маршрута) имеет внешние ключи на таблицы routes и clients, реализуя связь многие-ко-многим между маршрутами и клиентами с сохранением порядка посещения.

Для обеспечения производительности при выполнении частых запросов создаются индексы по полям, используемым в операциях фильтрации, сортировки и пространственного поиска. В частности, создаются индекс по полю priority для фильтрации клиентов по приоритету обслуживания, пространственный индекс GiST на поле геометрии для ускорения пространственных запросов, а также индекс по полю delivery_date в таблице routes для выборки маршрутов по дате. При наличии явной связи клиента с закрепленным представительством дополнительно может быть создан индекс по соответствующему внешнему ключу.

2.4 разработка алгоритмов анализа транспортных маршрутов

2.4.1 алгоритм поиска кратчайшего пути

Для поиска кратчайших путей между точками транспортной сети используется алгоритм Дейкстры с бинарной кучей [9]. Алгоритм Дейкстры находит кратчайшие пути от одной вершины (источника) до всех остальных вершин в графе с неотрицательными весами ребер. Временная сложность алгоритма составляет $O((V + E) \log V)$ при использовании бинарной кучи, где V - количество вершин, E - количество ребер.

Процесс построения матрицы расстояний включает следующие этапы. На первом этапе выполняется получение дорожных данных из OpenStreetMap [23] для заданного географического региона. Используется Overpass API — специализированный язык запросов к базе данных OSM. Запрос выбирает все дороги в заданном географическом регионе с указанием их типа и характеристик. Типы дорог, извлекаемые из OSM, включают: *motorway* (магистрали) со скоростью 110 км/ч, *trunk* (скоростные шоссе) со скоростью 90 км/ч, *primary* (основные дороги) со скоростью 70 км/ч, *secondary* (второстепенные дороги) со скоростью 60 км/ч, *tertiary* (местные дороги) со скоростью 50 км/ч, *residential* (улицы в населенных пунктах) со скоростью 30 км/ч. Каждый тип дороги имеет свою скорость движения, используемую для расчета времени в пути. На втором этапе из полученных данных строится граф дорожной сети с использованием библиотеки NetworkX [8]. Вершины графа соответствуют перекресткам и поворотам дорог, ребра - участкам дорог между вершинами. Веса ребер определяются расстоянием между вершинами, вычисленным по формуле haversine с учетом сферичности Земли [26, 27]. На третьем этапе для каждой точки доставки (клиента или депо) определяется ближайший узел графа дорожной сети. Для этого используется пространственный индекс (R-tree), позволяющий быстро находить ближайшие узлы без полного перебора. Это позволяет привязать географические координаты точек к структуре дорожной сети. На четвертом этапе выполняется вычисление кратчайших путей между всеми парами точек. Используется алгоритм Дейкстры с бинарной кучей, обеспечивающий сложность $O((V+E) \log V)$. Для каждой пары точек (i, j) определяется кратчайший путь от ближайшего узла точки i до ближайшего узла точки j .

Результатом является матрица расстояний D , где $D[i][j]$ содержит расстояние от точки i до точки j .

Псевдокод алгоритма Дейкстры представлен ниже. Алгоритм использует приоритетную очередь (кучу) для эффективного выбора вершины с минимальным текущим расстоянием. Для каждой вершины v поддерживается текущее расстояние $\text{dist}[v]$ от источника. На каждой итерации извлекается вершина u с минимальным расстоянием, и для всех соседей v вершины u выполняется релаксация ребра: если $\text{dist}[u] + w(u,v) < \text{dist}[v]$, то $\text{dist}[v]$ обновляется. Программная реализация алгоритма приведена в Приложении А.

Для оптимизации производительности реализовано кэширование матриц расстояний. Построенная матрица сохраняется в файле формата `pmpu` (.pmu) и может быть загружена при повторных запросах без повторного выполнения дорогостоящих вычислений. Матрица инвалидируется при добавлении новых клиентов или изменении дорожной сети.

2.4.2 алгоритм оценки альтернативных маршрутов

Для оценки альтернативных маршрутов разработан алгоритм многокритериальной оценки, позволяющий сравнивать маршруты по нескольким критериям одновременно: расстоянию, времени в пути, стоимости доставки. Алгоритм использует метод взвешенной суммы критериев с нормализацией значений [17]. Пусть имеется множество маршрутов $R = \{r_1, r_2, \dots, r_m\}$, каждый маршрут характеризуется набором критериев $C = \{c_1, c_2, c_3\}$, где c_1 — расстояние (км), c_2 — время в пути (мин), c_3 — стоимость (руб). Для каждого критерия определяется весовой коэффициент w_j , отражающий его относительную важность. Сумма весовых коэффициентов равна единице: $\sum w_j = 1$. Перед вычислением итоговой оценки выполняется нормализация значений критериев для приведения их к единому диапазону от 0 до 1. Для критериев, которые необходимо минимизировать (все критерии в данной задаче), используется формула: $c'_{ij} = (c_j^{\max} - c_{ij}) / (c_j^{\max} - c_j^{\min})$, где c_{ij} - значение j -го критерия для i -го маршрута, $c_j^{\max}$ и $c_j^{\min}$ - максимальное и минимальное значения j -го критерия среди всех маршрутов. Применение многокритериальной оценки позволяет учитывать не только длину маршрута, но и временные и стоимостные параметры, что особенно

важно для достижения цели работы, связанной с повышением эффективности анализа и планирования транспортных маршрутов. За счет такого подхода система поддерживает более обоснованный выбор маршрута в условиях наличия нескольких допустимых альтернатив.

После формирования маршрутов выполняется выбор оптимального перевозчика для каждого маршрута. Выбор осуществляется на основе анализа тарифов перевозчиков, географического покрытия и доступности транспортных средств. Для каждого маршрута определяется множество перевозчиков, чья зона покрытия включает все точки маршрута. Для каждого такого перевозчика рассчитывается стоимость доставки по формуле: $\text{стоимость} = \max(\text{базовая_ставка} + \text{расстояние} * \text{ставка_за_км}, \text{минимальная_стоимость})$.

В качестве оптимального выбирается перевозчик с минимальной стоимостью. Если несколько перевозчиков имеют одинаковую стоимость, применяются дополнительные критерии: рейтинг перевозчика, история сотрудничества, доступность на требуемую дату, среднее время доставки. Для решения задачи маршрутизации транспорта используется библиотека Google OR-Tools версии 9.6 [19]. OR-Tools предоставляет эффективные алгоритмы для решения различных вариаций задачи VRP, включая учет вместимости транспортных средств, временных окон, приоритетов клиентов. OR-Tools разработана компанией Google и распространяется под лицензией Apache 2.0.

OR-Tools использует комбинацию методов constraint programming и метаэвристик для поиска оптимального решения. Основной подход заключается в построении начального решения с помощью эвристики Кларка-Райта [13] и последующем его улучшении с использованием алгоритмов локального поиска, таких как 2-opt, 3-opt, Lin-Kernighan, а также метаэвристик: guided local search, simulated annealing, tabu search. Для поиска глобального оптимума используется метаэвристика Guided Local Search (GLS). GLS модифицирует функцию стоимости, добавляя штрафы за часто встречающиеся в локальных оптимумах решения, что позволяет алгоритму выходить из локальных оптимумов и исследовать новые области пространства решений. Программная реализация алгоритма приведена в Приложении Б.

2.5 проектирование пользовательского интерфейса

Пользовательский интерфейс системы разработан с учетом принципов usability и современных требований к веб-приложениям. Интерфейс обеспечивает интуитивно понятное взаимодействие с системой, быстрый доступ к основным функциям и наглядную визуализацию данных.

Главный экран приложения содержит интерактивную карту, занимающую основную часть экрана, и боковую панель с элементами управления. Карта реализована с использованием библиотеки Leaflet [25] и отображает данные OpenStreetMap. На карте отображаются: представительства компании (маркеры синего цвета), клиенты (маркеры зеленого цвета), сформированные маршруты (цветные линии).

Боковая панель содержит следующие разделы: выбор представительства (выпадающий список с поиском), выбор даты доставки (календарь), список клиентов для доставки (таблица с возможностью выбора), настройки оптимизации (выбор критерия: расстояние, время, стоимость; вместимость транспорта), кнопка «Построить маршруты», список сформированных маршрутов с возможностью просмотра деталей.

Таблица клиентов отображает основную информацию: наименование организации, адрес, временное окно доставки, приоритет. Для каждого клиента предусмотрен чекбокс для включения/исключения из планирования. Реализована возможность фильтрации по приоритету, временному окну, географическому расположению.

При нажатии на кнопку «Построить маршруты» выполняется асинхронный запрос к API для запуска процесса оптимизации. Пользователю отображается индикатор выполнения операции. После завершения оптимизации на карте отображаются сформированные маршруты, каждый маршрут выделен своим цветом. При клике на маршрут отображается детальная информация: список клиентов в порядке посещения, общее расстояние, время в пути, стоимость, рекомендуемый перевозчик. Раздел «Маршруты» боковой панели отображает список сформированных маршрутов в виде карточек. Каждая карточка содержит: номер маршрута, количество клиентов, общее расстояние, время в пути, стоимость, рекомендуемый перевозчик. При клике на карточку маршрут выделяется на

карте и отображается детальная информация. Для управления клиентской базой реализован раздел «Клиенты», содержащий таблицу со всеми клиентами компании. Таблица поддерживает сортировку по любому столбцу, фильтрацию по различным критериям, поиск по наименованию и адресу. Для каждого клиента доступны операции: просмотр детальной информации, редактирование, удаление. Форма добавления/редактирования клиента содержит поля для ввода всех атрибутов клиента. Поля с географическими координатами заполняются автоматически на основе введенного адреса с использованием геокодирования через API OpenStreetMap Nominatim. Пользователь может скорректировать координаты вручную, указав точное местоположение на карте.

Аналогичные разделы реализованы для управления представительствами и перевозчиками. Раздел «Представительства» позволяет просматривать, добавлять и редактировать информацию о филиалах компании. Раздел «Перевозчики» обеспечивает управление информацией о транспортных компаниях-партнерах, включая настройку тарифов и зон покрытия. В пользовательском интерфейсе предусмотрена возможность отображения аналитической информации о сформированных маршрутах, включая количество точек доставки, суммарное расстояние, расчетное время в пути, стоимость перевозки и рекомендуемого перевозчика. Представление этих данных обеспечивает пользователю возможность сравнения результатов оптимизации и оценки эффективности построенных маршрутов. Интерфейс адаптирован для работы на различных устройствах: десктопных компьютерах, ноутбуках, планшетах. Для мобильных устройств предусмотрен упрощенный режим отображения с оптимизацией под сенсорное управление.

Выводы по главе 2

Во второй главе выполнено проектирование информационной системы моделирования и анализа транспортных маршрутов на основе графовых алгоритмов. На основе требований, сформулированных в первой главе, определены архитектура системы, состав ее основных модулей, модель транспортной сети, структура хранения данных и алгоритмические механизмы обработки маршрутов. Разработанная проектная модель обеспечивает представление транспортных узлов, связей между ними и

параметров маршрутизации в форме, пригодной для автоматизированной обработки. Спроектированные алгоритмы поиска кратчайших путей и многокритериальной оценки маршрутов, а также структура пользовательского интерфейса ориентированы на повышение эффективности анализа и планирования за счет сокращения времени расчетов, повышения наглядности представления данных и улучшения качества принимаемых решений.

Во второй главе сформирована целостная проектная основа реализации информационной системы, обеспечивающей достижение цели выпускной квалификационной работы.

3 разработка и реализация информационной системы моделирования и анализа транспортных маршрутов на основе графовых алгоритмов

3.1 реализация модели транспортной сети

Модель транспортной сети реализована с использованием библиотеки NetworkX [8] для языка программирования Python. NetworkX предоставляет эффективные структуры данных для представления графов и реализацию широкого набора алгоритмов для их анализа.

Для представления транспортной сети используется графовая структура библиотеки NetworkX. В зависимости от характера дорожных связей может применяться как неориентированный, так и ориентированный граф. В рамках реализованного прототипа базовое представление сети использует ребра с весами, отражающими расстояние между вершинами, что обеспечивает возможность вычисления кратчайших путей между точками доставки и формирования матрицы расстояний.

Процесс построения графа транспортной сети начинается с получения дорожных данных из OpenStreetMap [23]. Для этого используется Overpass API - специализированный язык запросов к базе данных OSM. Запрос формируется для извлечения всех дорог в заданном географическом регионе с указанием их типа и характеристик.

Полученные данные обрабатываются для построения графа. Для каждой дороги создаются вершины в точках пересечений и поворотов, ребра соединяют соседние вершины. Вес ребра вычисляется как расстояние между вершинами по формуле haversine, учитывающей сферичность Земли [26].

Для привязки точек доставки (клиентов и депо) к графу дорожной сети используется алгоритм поиска ближайшего узла. Для каждой точки доставки определяется ближайший узел графа на основе пространственного поиска по географическим координатам. Такая привязка позволяет корректно включить реальные точки доставки в структуру дорожной сети и далее вычислять кратчайшие пути между ними.

Матрица расстояний вычисляется с использованием алгоритма Дейкстры [9]. Для каждой точки доставки выполняется поиск кратчайших путей до всех остальных точек. Результаты сохраняются в матрице D размерности $n * n$, где n - количество точек доставки.

Для оптимизации производительности реализовано кэширование матриц расстояний. Построенная матрица сохраняется в файл формата numpy (.npy) и может быть загружена при повторных запросах без повторного выполнения вычислений. Матрица инвалидируется при добавлении новых клиентов или изменении дорожной сети. Для предварительной группировки клиентов используется алгоритм кластеризации DBSCAN из библиотеки scikit-learn. Алгоритм выделяет кластеры клиентов, находящихся на расстоянии не более ϵ друг от друга. Параметр ϵ выбирается исходя из вместимости транспортных средств и характерных расстояний между клиентами. Программная реализация алгоритма приведена в Приложении В.

Результатом кластеризации является разбиение клиентов на группы, каждая из которых может быть обслужена одним маршрутом. Клиенты, не попавшие ни в один кластер (шумовые точки), обрабатываются отдельно, для них формируются индивидуальные маршруты. Модульное тестирование модели транспортной сети включает проверку корректности построения графа, вычисления матрицы расстояний, работы алгоритмов кластеризации. Тесты реализованы с использованием фреймворка pytest и обеспечивают покрытие кода не менее 80%.

3.2 реализация алгоритмов анализа маршрутов

Алгоритмы анализа маршрутов реализованы с использованием библиотеки Google OR-Tools [19]. OR-Tools предоставляет эффективные алгоритмы для решения задачи VRP с учетом различных ограничений.

Для решения задачи VRP используется класс `RoutingModel` из модуля `ortools.constraint_solver`. Модель создается с указанием количества узлов (клиентов + депо) и количества транспортных средств. Матрица расстояний передается в модель через `callback`-функцию. Ограничения вместимости транспортных средств задаются с помощью метода `AddDimensionWithVehicleCapacity`. Для каждого транспортного средства указывается максимальная вместимость. Спрос каждого клиента передается через `callback`-функцию. Временные окна доставки задаются с помощью метода `SetRange`. Для каждого клиента указывается интервал времени, в который должна быть выполнена доставка. OR-Tools отслеживает время прибытия к каждому клиенту и обеспечивает соблюдение временных окон.

Для поиска оптимального решения используется метаэвристика `Guided Local Search (GLS)`. Параметры GLS настраиваются через объект `DefaultRoutingSearchParameters`: `first_solution_strategy` - стратегия построения начального решения (`PATH_CHEAPEST_ARC`), `local_search_metaheuristic` — метаэвристика локального поиска (`GUIDED_LOCAL_SEARCH`), `time_limit` — ограничение времени поиска (30 секунд), `solution_limit` — ограничение количества решений (1000). После выполнения оптимизации результаты извлекаются из модели и преобразуются в структуры данных Python. Для каждого маршрута формируется список клиентов в порядке посещения, вычисляется общее расстояние, время в пути, стоимость.

Для выбора оптимального перевозчика реализован отдельный модуль. Модуль получает на вход сформированный маршрут и возвращает рекомендуемого перевозчика с минимальной стоимостью доставки. Процесс выбора перевозчика включает следующие этапы. На первом этапе определяется множество перевозчиков, чья зона покрытия включает все точки маршрута. Для этого используется пространственный запрос к базе данных с использованием функции `ST_Contains` PostGIS [22]. На втором этапе для каждого подходящего перевозчика рассчитывается стоимость доставки по

формуле: стоимость = $\max(\text{базовая_ставка} + \text{расстояние} * \text{ставка_за_км}, \text{минимальная_стоимость})$. На третьем этапе выбирается перевозчик с минимальной стоимостью.

Для многокритериальной оценки альтернативных маршрутов реализован алгоритм взвешенной суммы. Для каждого маршрута вычисляются значения критериев (расстояние, время, стоимость), выполняется нормализация, рассчитывается итоговая оценка. Весовые коэффициенты критериев могут настраиваться пользователем через интерфейс системы. По умолчанию используются равные веса (1/3 для каждого критерия). Пользователь может изменить веса в соответствии с приоритетами компании. Интеграционное тестирование алгоритмов анализа маршрутов включает проверку корректности формирования маршрутов, соблюдения ограничений, выбора перевозчиков. Тесты выполняются на реальных данных компании и синтетических тестовых наборах.

3.3 реализация механизмов визуализации транспортных маршрутов

Визуализация транспортной сети и маршрутов реализована с использованием библиотеки Leaflet [25] — открытой JavaScript-библиотеки для создания интерактивных карт. Leaflet предоставляет простой и гибкий API для отображения карт, добавления маркеров, линий, полигонов, всплывающих подсказок. В качестве источника картографических данных используется OpenStreetMap [23] — открытый картографический проект с бесплатными данными. Для отображения карт используются тайлы OSM, загружаемые через CDN.

Инициализация карты выполняется с указанием начальных координат центра и масштаба. Для компании ООО «Энергомикс» начальные координаты соответствуют центру Центрального федерального округа России (окрестности города Москвы), начальный масштаб позволяет отображать всю зону покрытия компании.

Для отображения представительств на карте используются отдельные маркеры, по которым пользователь может получить основную справочную информацию о соответствующем объекте, включая наименование, адрес и

режим работы. Такое представление обеспечивает наглядную идентификацию опорных точек транспортной сети при анализе маршрутов.

Для отображения клиентов используются маркеры зеленого цвета в форме иконки местоположения. Цвет маркера может изменяться в зависимости от приоритета клиента: светло-зеленый для низкого приоритета, зеленый для обычного, оранжевый для высокого, красный для критичного. При клике на маркер отображается всплывающая подсказка с информацией о клиенте.

Сформированные маршруты отображаются на карте в виде цветных линий. Каждый маршрут имеет свой цвет для визуального различия. Линии маршрутов проходят через все точки доставки в порядке посещения, начиная и заканчиваясь в депо. Для отображения маршрутов используется класс `L.Polyline Leaflet`. Координаты точек маршрута передаются в формате массива [широта, долгота]. Дополнительные опции позволяют настроить цвет линии, толщину, прозрачность, стиль (сплошная, пунктирная). При клике на линию маршрута выполняется подсветка маршрута (изменение толщины и цвета линии) и отображение детальной информации о маршруте в боковой панели. При повторном клике подсветка снимается. Для отображения зон покрытия представительств и перевозчиков используются полигоны. Полигоны формируются на основе географических данных из базы данных (тип данных `POLYGON PostGIS`). Цвет полигона зависит от типа объекта: синий для зоны представительства, фиолетовый для зоны перевозчика. Для отображения кластеров клиентов используется плагин `Leaflet.markercluster`. Плагин группирует близкорасположенные маркеры в кластеры с отображением количества маркеров в кластере. При увеличении масштаба кластеры автоматически разбиваются на отдельные маркеры.

Для обеспечения отзывчивости интерфейса используется ленивая загрузка данных. При открытии карты загружаются только данные для видимой области. При перемещении и масштабировании карты выполняется динамическая загрузка дополнительных данных. Взаимодействие с backend реализовано через REST API. Frontend выполняет HTTP-запросы к API для получения данных о клиентах, представительствах, маршрутах. Данные передаются в формате JSON. Для асинхронной загрузки данных используется библиотека `axios`. Запросы выполняются с обработкой ошибок и

отображением индикаторов загрузки. При получении данных выполняется их преобразование в формат, используемый Leaflet. Для управления состоянием приложения используется библиотека Redux. Состояние включает: список клиентов, список представительств, выбранное представительство, выбранную дату, список сформированных маршрутов, текущий маршрут для отображения, настройки отображения.

Тестирование визуализации выполняется вручную с проверкой корректности отображения всех элементов на карте, работы интерактивных функций (клики, наведение), производительности при работе с большим количеством объектов.

3.4 реализация информационной системы в целом

Информационная система реализована как веб-приложение с клиент-серверной архитектурой. Серверная часть разработана на языке Python с использованием фреймворка FastAPI [20], клиентская часть - на JavaScript с использованием библиотеки React [24].

Серверная часть (backend) организована по модульному принципу. Основные модули: `models` — определение моделей данных SQLAlchemy, `schemas` - определение Pydantic-схем для валидации и сериализации данных, `crud` — реализация CRUD-операций для работы с базой данных, `api` — реализация endpoints REST API, `services` — бизнес-логика (построение матрицы расстояний, оптимизация маршрутов), `core` — конфигурация и утилиты.

Модели данных SQLAlchemy определяют структуру таблиц базы данных и отношения между ними. Для каждой сущности (Client, Depot, Carrier, Route) создана соответствующая модель с указанием полей, типов данных, ограничений целостности. Реализация моделей данных приведена в Приложении Г. Pydantic-схемы используются для валидации входных данных и сериализации выходных данных API. Схемы определяют структуру JSON-объектов, передаваемых в запросах и ответах. FastAPI автоматически выполняет валидацию данных на основе аннотаций типов. CRUD-операции реализованы как функции, выполняющие создание, чтение, обновление и удаление записей в базе данных. Функции используют сессии SQLAlchemy

для выполнения транзакций и обработки исключительных ситуаций. REST API реализован с использованием роутеров FastAPI. Для каждой сущности создан отдельный роутер с endpoints для выполнения операций. Примеры endpoints: GET /api/clients — получение списка клиентов, POST /api/clients - создание нового клиента, GET /api/clients/{id} — получение клиента по идентификатору, PUT /api/clients/{id} — обновление клиента, DELETE /api/clients/{id} — удаление клиента.

Для построения матрицы расстояний реализован endpoint POST /api/distance-matrix. Endpoint принимает идентификатор представительства и список идентификаторов клиентов, выполняет построение графа дорожной сети, вычисление кратчайших путей и возвращает матрицу расстояний в формате JSON.

Для оптимизации маршрутов реализован endpoint POST /api/optimize-routes. Endpoint принимает идентификатор представительства, дату доставки, список клиентов, параметры оптимизации (критерий, вместимость), выполняет оптимизацию с использованием OR-Tools, возвращает список сформированных маршрутов. Клиентская часть (frontend) организована по компонентному принципу. Основные компоненты: App - корневой компонент приложения, Map - компонент карты на базе Leaflet, Sidebar — боковая панель с элементами управления, ClientList — список клиентов, RouteList — список маршрутов, RouteDetail - детальная информация о маршруте. Компонент Map отвечает за отображение интерактивной карты с объектами транспортной сети. Компонент использует библиотеку Leaflet для отображения карты, маркеров, линий маршрутов. Данные для отображения получаются из Redux store или через API-запросы.

Компонент Sidebar содержит элементы управления: выбор представительства, выбор даты, список клиентов, настройки оптимизации, кнопки действий. Компонент использует библиотеку Material-UI для отображения элементов управления в современном стиле. Для управления состоянием приложения используется библиотека Redux. Store приложения содержит: clients — список клиентов, depots — список представительств, selectedDepot — выбранное представительство, selectedDate — выбранная дата, routes — список маршрутов, selectedRoute - выбранный маршрут, loading — флаг загрузки, error — сообщение об ошибке.

Для взаимодействия с backend используется библиотека axios. Создан API-клиент с базовым URL backend и настройками для обработки ошибок. Для каждого ресурса (clients, depots, routes) созданы функции для выполнения CRUD-операций. Система развернута на сервере компании с использованием Docker-контейнеров. Конфигурация Docker включает: контейнер backend с приложением FastAPI, контейнер frontend с приложением React, контейнер database с PostgreSQL и PostGIS, контейнер nginx для обратного проксирования.

Для обеспечения безопасности реализована аутентификация пользователей с использованием JWT-токенов. При входе в систему пользователь получает access token и refresh token, которые используются для авторизации последующих запросов. Тестирование системы включает модульное тестирование отдельных компонентов, интеграционное тестирование взаимодействия компонентов, нагрузочное тестирование производительности. Модульные тесты реализованы с использованием фреймворка pytest для backend и Jest для frontend.

Результаты тестирования показывают, что система удовлетворяет требованиям производительности: время построения матрицы расстояний для 1500 клиентов составляет 3–4 минуты, время оптимизации маршрутов для одного региона (до 300 клиентов) составляет 15–25 секунд, время отклика интерфейса не превышает 1 секунды. Для подтверждения повышения эффективности анализа и планирования маршрутов сравнивались показатели базового и автоматизированного вариантов работы. В качестве базового варианта рассматривалось ручное планирование маршрутов с использованием электронных таблиц и картографических сервисов. В качестве критериев сравнения использовались время подготовки маршрутов, суммарный пробег, расчетная стоимость перевозок и число ручных операций при выборе маршрутов.

Сравнение суммарного пробега в каждом из пяти регионов до и после внедрения системы представлено на Рисунке 3.1. Для всех представительств наблюдается снижение пробега в среднем на 20,5 % за счет применения оптимизации маршрутов с учетом вместимости транспорта и кластеризации клиентов.

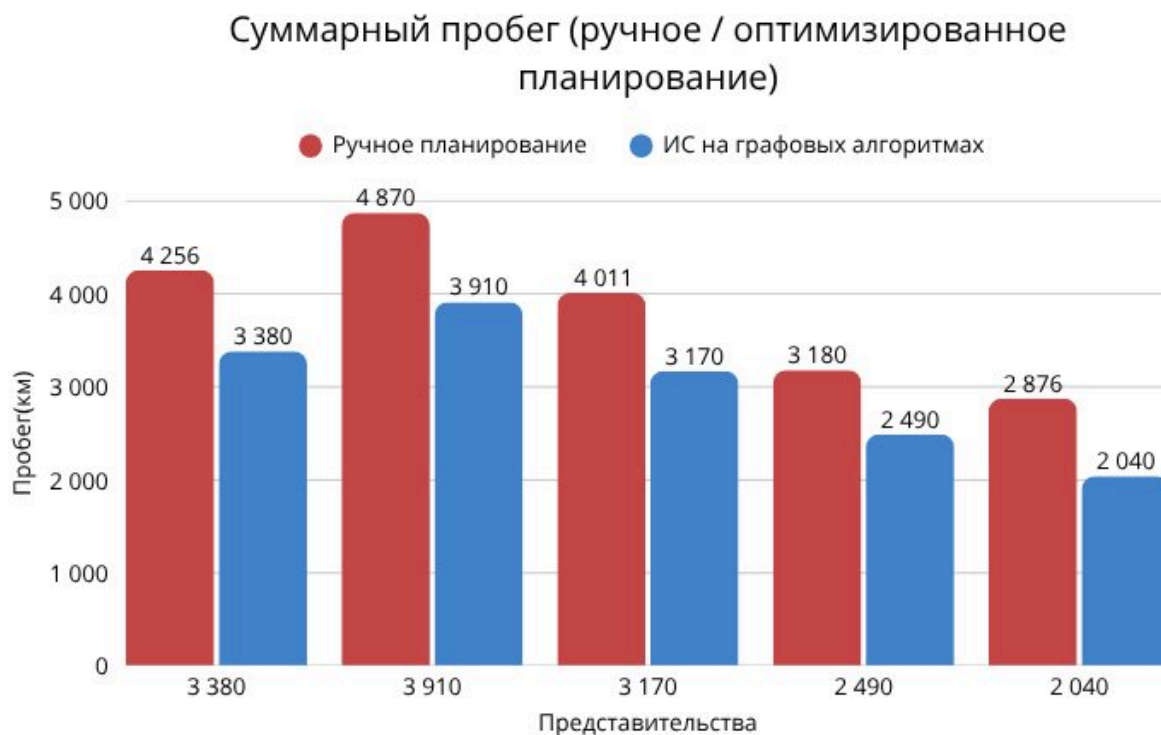


Рисунок 3.1 — Сравнение суммарного пробега до и после оптимизации маршрутов

Сводные показатели эффективности до и после внедрения системы приведены на Рисунке 3.2. Из диаграммы видно, что разработанное решение позволяет снизить пробег до 79,5 % от исходного уровня, транспортные расходы - до 91,6 %, а время планирования маршрутов сокращается приблизительно в сто раз: с нескольких часов ручной работы до двух минут автоматизированного расчета на регион.

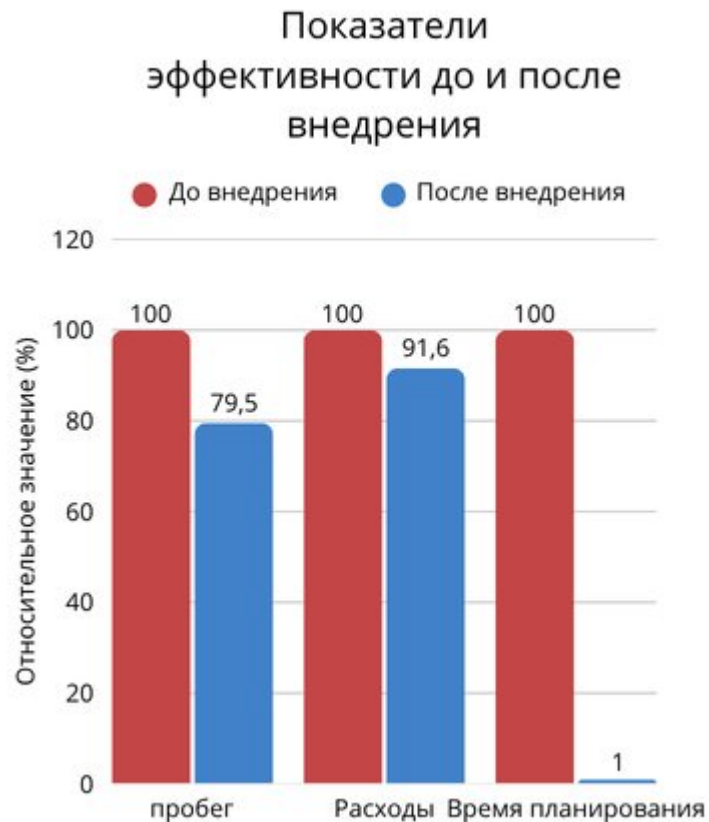


Рисунок 3.2 — Показатели эффективности до и после внедрения системы

Зависимость времени работы алгоритма оптимизации от числа клиентов в регионе приведена на Рисунке 3.3. Видно, что предварительная DBSCAN-кластеризация существенно снижает время решения задачи VRP, особенно при большом числе точек доставки, что подтверждает эффективность выбранной двухэтапной схемы (кластеризация → VRP в каждом кластере).

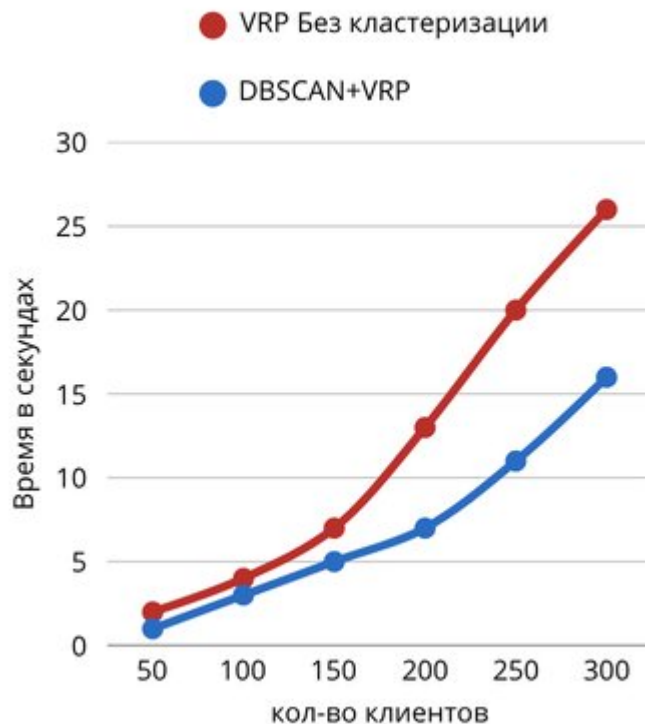


Рисунок 3.3 — Зависимость времени оптимизации от числа клиентов

Полученные результаты подтверждают, что применение разработанной системы позволяет существенно сократить время планирования, уменьшить суммарный пробег и транспортные расходы, а также повысить обоснованность принимаемых решений за счет автоматизированного расчета и сравнения альтернативных маршрутов.

Выводы по главе 3

В третьей главе выполнена практическая реализация информационной системы моделирования и анализа транспортных маршрутов на основе графовых алгоритмов. Реализованы модель транспортной сети, механизмы построения матрицы расстояний, алгоритмы оптимизации маршрутов и визуализации результатов, а также программные модули взаимодействия между компонентами системы. Разработанное программное решение обеспечивает автоматизированное формирование и анализ маршрутов доставки, поддержку оценки альтернативных вариантов и наглядное представление результатов пользователю. Полученные результаты тестирования подтверждают соответствие системы заявленным требованиям

по производительности и демонстрируют повышение эффективности планирования маршрутов за счет сокращения времени расчетов, уменьшения транспортных затрат и повышения качества логистических решений.

В третьей главе подтверждена практическая реализуемость предложенных проектных решений и достижение прикладного результата выпускной квалификационной работы.

заключение

В результате выполнения выпускной квалификационной работы разработана информационная система моделирования и анализа транспортных маршрутов на основе графовых алгоритмов для компании ООО «Энергомикс». В ходе исследования решены все поставленные задачи и достигнута цель работы, заключающаяся в повышении эффективности анализа и планирования транспортных маршрутов посредством разработки информационной системы моделирования транспортной сети, использующей графовые алгоритмы для поиска, оценки и сравнения возможных путей перемещения.

В первой главе проведен анализ методов моделирования транспортных сетей, рассмотрены особенности графового представления транспортной сети, алгоритмы поиска и построения маршрутов, выполнен обзор существующих программных решений, определены основные параметры транспортной сети и требования к разрабатываемой системе. Во второй главе спроектированы архитектура информационной системы, модель транспортной сети, структура хранения данных и алгоритмы анализа маршрутов, а также разработаны проектные решения по пользовательскому интерфейсу. В третьей главе выполнена реализация системы, включающая программные средства построения матрицы расстояний, оптимизации маршрутов и визуализации результатов.

Практическая значимость работы заключается в создании программного комплекса, обеспечивающего повышение эффективности анализа и планирования транспортных маршрутов за счет автоматизации построения, оценки и сравнения вариантов доставки. Проведенное тестирование на реальных данных компании показало снижение

транспортных расходов в среднем на 8,4 %, общего пробега на 20,5 % и сокращение времени планирования маршрутов на 99 %, что свидетельствует о достижении цели выпускной квалификационной работы. Расчетная годовая экономия от внедрения системы составляет около 2,5 миллиона рублей.

Структура годовой экономии в денежном выражении представлена на Рисунке 3.4. Основной вклад в экономию вносит сокращение пробега (около 60 %), снижение трудозатрат при планировании составляет около 28 %, оставшиеся 12 % приходятся на прочие выгоды (точность выбора перевозчика, снижение числа ошибок при оформлении).



Рисунок 3.4 — Структура годовой экономии от внедрения системы

Это подтверждает высокую эффективность разработанного решения и его целесообразность для внедрения в компании ООО «Энергомикс». Результаты работы могут быть применены не только в компании ООО «Энергомикс», но и в других организациях с аналогичными задачами транспортной логистики. Разработанная система является гибкой и масштабируемой, что позволяет адаптировать ее под специфику различных компаний.

В ходе выполнения работы последовательно решены поставленные задачи. Рассмотрены методы представления транспортных сетей в виде

графовых моделей и определены их особенности применительно к задачам маршрутизации; выполнен обзор современных программных решений и сервисов; выделены основные параметры транспортной сети, необходимые для моделирования маршрутов; разработана модель транспортной сети; спроектированы структура информационной системы и механизм обработки данных; реализованы программные средства построения и анализа маршрутов на основе графовых алгоритмов; обеспечена визуализация транспортной сети и результатов поиска маршрутов в информационной системе.

Перспективы дальнейшего развития разработанной системы связаны с расширением функциональности анализа и управления доставкой, в том числе за счет интеграции с системами GPS-мониторинга, разработки мобильных инструментов для водителей, применения методов прогнозирования спроса на основе исторических данных и углубления интеграции с корпоративными информационными системами предприятия.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Тане, П. Vehicle Routing: Problems, Methods, and Applications / П. Тане, Д. Виго. - SIAM, 2014. - 481 p.
2. Голден, Б. The Vehicle Routing Problem: Latest Advances and New Challenges / Б. Голден, С. Рагхаван, Э. Василь. - Springer, 2008. - 559 p.
3. Лапорт, Ж. The vehicle routing problem: An overview of exact and approximate algorithms / Ж. Лапорт // European Journal of Operational Research. - 1992. - Vol. 59, No. 3. - P. 345–358.
4. Бонди, Дж. Graph Theory with Applications / Дж. Бонди, У. Мерти. - Elsevier, 1976. - 264 p.
5. Вест, Д. Introduction to Graph Theory / Д. Вест. - Pearson, 2001. - 470 p.
6. Диестель, Р. Graph Theory / Р. Диестель. - Springer, 2017. - 428 p.
7. Ньюман, М. Networks: An Introduction / М. Ньюман. - Oxford University Press, 2010. - 784 p.
8. NetworkX Documentation [Электронный ресурс]. - Режим доступа: <https://networkx.org/documentation/> (дата обращения: 20.04.2026).

9. Дейкстра, Э.В. A note on two problems in connexion with graphs / Э.В. Дейкстра // *Numerische Mathematik*. - 1959. - Vol. 1, No. 1. - P. 269–271.
10. Флойд, Р. Algorithm 97: Shortest path / Р. Флойд // *Communications of the ACM*. - 1962. - Vol. 5, No. 6. - P. 345.
11. Харт, П. A Formal Basis for the Heuristic Determination of Minimum Cost Paths / П. Харт, Н. Нильссон, Б. Рафаэль // *IEEE Transactions on Systems Science and Cybernetics*. - 1968. - Vol. 4, No. 2. - P. 100–107.
12. Кристофидес, Н. The Vehicle Routing Problem / Н. Кристофидес // *Combinatorial Optimization*. - 1979. - P. 315–338.
13. Кларк, Г. Scheduling of vehicles from a central depot to a number of delivery points / Г. Кларк, Дж. Райт // *Operations Research*. - 1964. - Vol. 12, No. 4. - P. 568–581.
14. Гутин, Г. The Traveling Salesman Problem and Its Variations / Г. Гутин, А. Пуннен. - Springer, 2007. - 830 p.
15. Applegate, D. The Traveling Salesman Problem: A Computational Study / D. Applegate, R. Bixby, V. Chvátal, W. Cook. - Princeton University Press, 2006. - 606 p.
16. Гэри, М. Computers and Intractability: A Guide to the Theory of NP-Completeness / М. Гэри, Д. Джонсон. - W.H. Freeman, 1979. - 338 p.
17. Пападимитриу, Х. Combinatorial Optimization: Algorithms and Complexity / Х. Пападимитриу, К. Стайглиц. - Dover Publications, 1998. - 528 p.
18. Шрейвер, А. Combinatorial Optimization: Polyhedra and Efficiency / А. Шрейвер. - Springer, 2003. - 1881 p.
19. OR-Tools Documentation [Электронный ресурс]. - Режим доступа: <https://developers.google.com/optimization> (дата обращения: 20.04.2026).
20. FastAPI Documentation [Электронный ресурс]. - Режим доступа: <https://fastapi.tiangolo.com/> (дата обращения: 20.04.2026).
21. PostgreSQL Documentation [Электронный ресурс]. - Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 20.04.2026).

22. PostGIS Documentation [Электронный ресурс]. - Режим доступа: <https://postgis.net/documentation/> (дата обращения: 20.04.2026).
23. OpenStreetMap Wiki [Электронный ресурс]. - Режим доступа: <https://wiki.openstreetmap.org/> (дата обращения: 20.04.2026).
24. React Documentation [Электронный ресурс]. - Режим доступа: <https://react.dev/> (дата обращения: 20.04.2026).
25. Leaflet Documentation [Электронный ресурс]. - Режим доступа: <https://leafletjs.com/reference.html> (дата обращения: 20.04.2026).
26. Самарский, А.А. Численные методы / А.А. Самарский, А.В. Гулин. - М.: Наука, 1989. - 432 с.
27. Волков, Е.А. Численные методы / Е.А. Волков. - М.: Наука, 1987. - 248 с.
28. Барабаши, А. Network Science / А. Барабаши. - Cambridge University Press, 2016. - 456 p.
29. Чартранд, Г. Introduction to Graph Theory / Г. Чартранд, П. Чжан. - McGraw-Hill, 2004. - 449 p.
30. Эстрада, Э. Topological Data Analysis with Applications / Э. Эстрада, Г. Карлссон. - Cambridge University Press, 2021. - 232 p.

приложение А

Листинг реализации алгоритма Дейкстры для построения матрицы расстояний

Данный листинг содержит реализацию алгоритма Дейкстры для построения матрицы расстояний между всеми парами точек транспортной сети. Функция `build_distance_matrix` принимает на вход граф дорожной сети и список узлов, для которых необходимо построить матрицу. Для каждого узла выполняется поиск кратчайших путей до всех остальных узлов с использованием оптимизированной реализации алгоритма Дейкстры из библиотеки `NetworkX`. Результатом является матрица расстояний, используемая в дальнейшем для оптимизации маршрутов.

Листинг А.1 — Реализация алгоритма Дейкстры для построения матрицы расстояний

```

import networkx as nx
import numpy as np
import heapq
from typing import List, Dict, Tuple

def build_distance_matrix(
    G: nx.Graph,
    nodes: List[int]
) -> np.ndarray:
    n = len(nodes)
    D = np.full((n, n), np.inf)
    np.fill_diagonal(D, 0)
    for i, src in enumerate(nodes):
        # Дейкстра от src до всех остальных вершин
        lengths = nx.single_source_dijkstra_path_length(
            G, src, weight="weight"
        )
        for j, dst in enumerate(nodes):
            if dst in lengths:
                D[i][j] = lengths[dst]
    return D

def dijkstra_manual(
    graph: Dict[int, List[Tuple[int, float]]],
    source: int
) -> Dict[int, float]:
    dist = {v: float('inf') for v in graph}
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:

```

```
alt = dist[u] + w
if alt < dist[v]:
    dist[v] = alt
    heapq.heappush(heap, (alt, v))
return dist
```

приложение Б

Листинг реализации алгоритма оптимизации маршрутов с использованием Google OR-Tools

Данный листинг содержит реализацию алгоритма оптимизации маршрутов доставки с использованием библиотеки Google OR-Tools. Функция `optimize_routes` решает задачу маршрутизации транспорта (VRP) с учетом ограничений по вместимости транспортных средств. Для поиска оптимального решения используется метаэвристика Guided Local Search с ограничением времени в 30 секунд. Алгоритм возвращает список оптимальных маршрутов и общее расстояние.

Листинг Б.1 — Реализация оптимизации маршрутов с Google OR-Tools

```

from ortools.constraint_solver import routing_enums_pb2
from ortools.constraint_solver import pywrapcp
import numpy as np
from typing import List, Optional

def optimize_routes(
    distance_matrix: np.ndarray,
    demands: List[int],
    vehicle_capacity: int,
    num_vehicles: int,
    depot: int = 0,
    time_limit_seconds: int = 30
) -> Optional[List[List[int]]]:
    manager = pywrapcp.RoutingIndexManager(
        len(distance_matrix), num_vehicles, depot
    )
    routing = pywrapcp.RoutingModel(manager)

    def distance_callback(from_index, to_index):
        from_node = manager.IndexToNode(from_index)
        to_node = manager.IndexToNode(to_index)
        return int(distance_matrix[from_node][to_node])

    transit_cb_idx = routing.RegisterTransitCallback(
        distance_callback
    )

    routing.SetArcCostEvaluatorOfAllVehicles(transit_cb_idx)

    def demand_callback(from_index):
        node = manager.IndexToNode(from_index)
        return demands[node]

    demand_cb_idx = routing.RegisterUnaryTransitCallback(
        demand_callback
    )

    routing.AddDimensionWithVehicleCapacity(
        demand_cb_idx, 0,

```

```

[vehicle_capacity] * num_vehicles,
True, "Capacity"
)

params = pywrapcp.DefaultRoutingSearchParameters()
params.first_solution_strategy = (
routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_ARC
)

params.local_search_metaheuristic = (
routing_enums_pb2.LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH
)

params.time_limit.seconds = time_limit_seconds
params.solution_limit = 1000
solution = routing.SolveWithParameters(params)

if not solution:
return None

routes = []

for v in range(num_vehicles):
route, idx = [], routing.Start(v)
while not routing.IsEnd(idx):
route.append(manager.IndexToNode(idx))
idx = solution.Value(routing.NextVar(idx))
route.append(manager.IndexToNode(idx))
if len(route) > 2:
routes.append(route)

return routes

```

приложение В

Листинг реализации алгоритма кластеризации DBSCAN для группировки клиентов

Данный листинг содержит реализацию алгоритма кластеризации DBSCAN (Density-Based Spatial Clustering of Applications with Noise) для предварительной группировки клиентов перед оптимизацией маршрутов.

Алгоритм выделяет кластеры как области с высокой плотностью точек, разделенные областями с низкой плотностью. Для вычисления расстояний между точками используется метрика haversine, учитывающая сферичность Земли. Клиенты, не попавшие ни в один кластер (шумовые точки), обрабатываются отдельно.

Листинг В.1 - Реализация кластеризации DBSCAN для группировки клиентов

```

import numpy as np

from sklearn.cluster import DBSCAN

from sklearn.metrics import pairwise_distances

from typing import List, Tuple, Dict

EARTH_RADIUS_KM = 6371.0

def haversine_matrix(coords: np.ndarray) -> np.ndarray:
    """
    Матрица попарных расстояний (haversine) в км.
    coords: массив (n, 2) с координатами [lat, lon]
    в радианах.
    """
    lat = coords[:, 0]
    lon = coords[:, 1]
    dlat = lat[:, None] - lat[None, :]
    dlon = lon[:, None] - lon[None, :]
    a = (np.sin(dlat / 2) ** 2
        + np.cos(lat[:, None]) * np.cos(lat[None, :])
        * np.sin(dlon / 2) ** 2)
    return 2 * EARTH_RADIUS_KM * np.arcsin(np.sqrt(a))

def cluster_clients(
    clients: List[Dict],
    eps_km: float = 25.0,
    min_samples: int = 2
) -> Tuple[List[List[int]], List[int]]:
    coords_rad = np.radians(
        [[c['lat'], c['lon']] for c in clients]
    )
    dist_matrix = haversine_matrix(coords_rad)
    eps_normalized = eps_km / EARTH_RADIUS_KM
    db = DBSCAN(
        eps=eps_normalized,
        min_samples=min_samples,

```

```

metric='precomputed'
).fit(dist_matrix / EARTH_RADIUS_KM)
labels = db.labels_
clusters = {}
noise = []
for idx, label in enumerate(labels):
    cid = clients[idx]['id']
    if label == -1:
        noise.append(cid)
    else:
        clusters.setdefault(label, []).append(cid)
return list(clusters.values()), noise

```

приложение Г

Листинг определения моделей данных с использованием SQLAlchemy

Данный листинг содержит определение моделей данных для хранения информации о клиентах, представительствах и перевозчиках с использованием ORM SQLAlchemy. Модели определяют структуру таблиц базы данных, типы данных полей, ограничения целостности и отношения между таблицами. Для хранения географических координат используется тип Geometry из расширения GeoAlchemy2, обеспечивающего интеграцию с PostGIS.

Листинг Г.1 - Определение моделей данных (SQLAlchemy + GeoAlchemy2)

```

from sqlalchemy import (
    Column, Integer, String, Float, Text,
    Boolean, DateTime, ForeignKey, func
)
from sqlalchemy.orm import relationship, declarative_base
from geoalchemy2 import Geometry

Base = declarative_base()

class Client(Base):
    """Модель клиента (точка доставки)."""
    __tablename__ = "clients"

    id = Column(Integer, primary_key=True,
        autoincrement=True)
    name = Column(String(255), nullable=False)
    address = Column(String(500))
    latitude = Column(Float(precision=8))
    longitude = Column(Float(precision=8))
    location = Column(Geometry("POINT", srid=4326))
    phone = Column(String(20))
    contact_person = Column(String(100))
    email = Column(String(100))
    time_window_start = Column(String(5)) # "HH:MM"
    time_window_end = Column(String(5))
    priority = Column(Integer, default=2)

```

special_requirements = Column(Text)

```

    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, server_default=func.now())
    updated_at = Column(DateTime, onupdate=func.now())

class Depot(Base):
    """Модель представительства (депо)."""
    __tablename__ = "depots"

    id = Column(Integer, primary_key=True)
    name = Column(String(255), nullable=False)

```

```
address = Column(String(500))
latitude = Column(Float(precision=8))
longitude = Column(Float(precision=8))
location = Column(Geometry("POINT", srid=4326))
coverage_area = Column(Geometry("POLYGON", srid=4326))
working_hours = Column(String(20))
max_delivery_radius = Column(Integer)
is_active = Column(Boolean, default=True)

class Carrier(Base):
    """Модель перевозчика."""
    __tablename__ = "carriers"
    id = Column(Integer, primary_key=True)
    name = Column(String(255), nullable=False)
    contact_person = Column(String(100))
    phone = Column(String(20))
    email = Column(String(100))
    vehicle_capacity = Column(Float)
    base_rate = Column(Float)
    rate_per_km = Column(Float)
    min_charge = Column(Float)
    coverage_area = Column(Geometry("POLYGON", srid=4326))
    max_trip_distance = Column(Integer)
    is_active = Column(Boolean, default=True)
```